

Programming Assignment 3 Report

Piecewise Polynomial Interpolation, Cubic Splines,
and Yield Curve Estimation

Godfred Antwi Koduah

Foundations of Computational Mathematics 2 — Spring 2026

April 15, 2026

1 Executive Summary

This report documents the design, implementation, and empirical validation of piecewise polynomial interpolation and cubic spline algorithms, together with their application to yield curve estimation from financial data. The C++ implementation encompasses five major components: a global Barycentric Form 1 interpolant using Chebyshev points (carried forward from Programming Assignment 2), a piecewise polynomial interpolant $g_d(x)$ supporting degrees $d = 1, 2, 3$ and Hermite cubic interpolation with both uniform and Chebyshev-2 interior points, a cubic spline using the second-derivative (moment) parameterization on nonuniform meshes (Spline Code 1), and a cubic spline using the cubic B-spline basis on uniform meshes (Spline Code 2). Both spline codes support natural boundary conditions (prescribed $s''(x_0)$ and $s''(x_n)$) and clamped boundary conditions (prescribed $s'(x_0)$ and $s'(x_n)$), with the clamped case requiring the derivation of two additional equations that relate the endpoint derivatives to the moment parameters.

The empirical validation in Task 1 demonstrates that both spline codes reproduce polynomials of degree three or less to machine precision ($\sim 10^{-14}$), that the two spline parameterizations produce identical results (differences at $\sim 10^{-16}$), and that the clamped spline converges at rate $O(h^4)$ while the natural spline converges at rate $O(h^2)$ for general functions. The convergence table comparing all methods reveals that the global Chebyshev interpolant achieves spectral (exponential) convergence for smooth functions, while the piecewise methods converge algebraically with rates governed by the polynomial degree.

Task 2 applies the natural cubic spline to eight yield curve data points (t_i, y_i) on a nonuniform mesh to estimate the yield $y(t)$, the instantaneous forward rate $f(t) = y(t) + ty'(t)$, and the discount factor $D(t) = e^{-ty(t)}$ at 40 equally-spaced maturities from $t = 0.5$ to $t = 20$ years. The investigation of piecewise polynomial alternatives reveals that standard piecewise polynomials (degrees 1, 2, or 3) cannot reliably compute $f(t)$ because their derivatives are discontinuous at knots, while the Hermite cubic piecewise polynomial provides C^1 continuity and allows $f(t)$ computation everywhere, though at the cost of requiring externally estimated derivatives at the data points.

2 Statement of the Problem

2.1 Background and Objectives

This assignment requires the implementation and systematic validation of piecewise polynomial interpolation methods and cubic splines, building on the global interpolation infrastructure from Programming Assignment 2. The transition from global to piecewise interpolation is motivated by the fundamental limitation of high-degree polynomial interpolation: while Chebyshev interpolation converges spectrally for analytic functions, the required degree grows with the number of data points, and convergence is not guaranteed for functions with limited smoothness. Piecewise methods address this by using low-degree polynomials on each subinterval, achieving convergence through mesh refinement (increasing n , the number of subintervals) rather than degree elevation (increasing d , the polynomial degree).

The central idea of piecewise interpolation is to sacrifice global polynomial degree for local flexibility: instead of a single degree- n polynomial that must thread through all $n + 1$ data points (potentially exhibiting Runge-like oscillations on uniform meshes), we use n polynomials of degree $d \leq 3$, each responsible for only a small portion of the domain. The trade-off is clear: piecewise linear interpolation converges only as $O(h^2)$ versus exponential convergence for Chebyshev interpolation of analytic functions, but piecewise methods work robustly for any continuous function and any mesh, without the need for specially chosen node distributions.

The cubic spline occupies a unique position in the hierarchy of piecewise methods: it uses the lowest polynomial degree (three) that permits C^2 continuity across the entire interpolation domain. Lower-degree piecewise polynomials ($d \leq 2$) cannot achieve C^2 continuity with the standard number of interpolation conditions. Higher-degree splines ($d \geq 4$) can achieve higher smoothness but introduce additional free parameters that require extra conditions to resolve. The cubic spline is therefore the “sweet spot” that balances smoothness, accuracy, and simplicity.

The assignment specifies five code components: (1) the global Barycentric Form 1 interpolant from Assignment 2, serving as a reference for comparison; (2) a piecewise polynomial interpolant $g_d(x)$ supporting degrees $d = 1, 2, 3$ and Hermite cubic, with both uniform and Chebyshev-2 interior points on each subinterval, implemented using local Newton divided differences; (3) Spline Code 1 using the s_i'' (moment) parameterization on nonuniform meshes; (4) Spline Code 2 using the cubic B-spline basis on uniform meshes; and (5) evaluation routines for all methods.

2.2 Required Boundary Conditions

Both spline codes must support two types of Hermite boundary conditions. The first type specifies the second derivatives at the endpoints, $(s''(x_0), s''(x_n))$, known as natural boundary conditions when both values are zero. The second type specifies the first derivatives at the endpoints, $(s'(x_0), s'(x_n))$, known as clamped (or complete) boundary conditions. Depending on the parameterization, one of these cases requires the derivation of additional equations. For the s_i'' parameterization used in Spline Code 1, the natural boundary conditions directly specify M_0 and M_n , while the clamped boundary conditions require two additional equations relating $s'(x_0)$ and $s'(x_n)$ to the moment parameters. For the B-spline parameterization in Spline Code 2, both boundary condition types require elimination of the auxiliary coefficients c_{-1} and c_{n+1} through substitution.

2.3 Task Descriptions

Task 1 requires empirical validation of all codes through carefully designed experiments. The validation must demonstrate polynomial reproduction (any polynomial of degree ≤ 3 must be reproduced exactly by the cubic spline), piecewise cubic reproduction (a piecewise cubic on the same mesh must be matched exactly), cross-comparison of the two spline codes, comparison of all methods (spline, piecewise polynomial, global interpolation) in terms of accuracy and computational complexity, verification of both boundary condition types, and robustness testing with randomly generated polynomials.

Task 2 applies the natural cubic spline to a specific yield curve dataset consisting of eight data points (t_i, y_i) on a nonuniform mesh, where $y(t)$ represents the zero-coupon yield as a function of maturity t . The relationships $D(t) = e^{-ty(t)}$ and $f(t) = y(t) + ty'(t)$ connect the yield to the discount factor and the instantaneous forward rate, respectively. The task requires tabulation of all three quantities at 40 maturities from $t = 0.5$ to $t = 20$ with increment $\Delta t = 0.5$, followed by an investigation of whether piecewise polynomial alternatives can replace the cubic spline for this application.

3 Description of the Mathematics

3.1 Piecewise Polynomial Interpolation

Given a partition $a = x_0 < x_1 < \dots < x_n = b$ of the interval $[a, b]$ into n subintervals $[x_i, x_{i+1}]$, a piecewise polynomial interpolant $g_d(x)$ of degree d is constructed by building an

independent polynomial of degree d on each subinterval. On the k -th subinterval $[x_k, x_{k+1}]$, the local polynomial $p_k(x)$ interpolates the function $f(x)$ at $d + 1$ points chosen within or on the boundary of $[x_k, x_{k+1}]$.

For degree $d = 1$ (piecewise linear), only the two endpoints x_k and x_{k+1} are used, giving the linear interpolant $p_k(x) = f(x_k) + \frac{f(x_{k+1}) - f(x_k)}{x_{k+1} - x_k}(x - x_k)$. This is C^0 (continuous) but not C^1 (the derivative has jump discontinuities at the knots). The reason the derivative is discontinuous is that on each subinterval, the slope is a constant $(f_{i+1} - f_i)/h_i$ determined solely by the two endpoint values, and there is no mechanism to ensure that adjacent slopes match. The approximation error is $O(h^2)$ where h is the maximum subinterval width, specifically $\|f - g_1\|_\infty \leq \frac{h^2}{8} \|f''\|_\infty$.

For degree $d = 2$ (piecewise quadratic), three points are needed: the two endpoints and one interior point. When using uniform interior points, the midpoint $(x_k + x_{k+1})/2$ is chosen. When using Chebyshev-2 points within $[x_k, x_{k+1}]$, the three nodes are the endpoints and the mapped midpoint $\cos(\pi/2) = 0$, which is the same as the midpoint—this is why Chebyshev-2 and uniform placement give identical results for $d = 2$, as confirmed empirically in Test 7. The approximation error is $O(h^3)$, which provides one additional order of accuracy over the linear case at the cost of one extra function evaluation per subinterval.

For degree $d = 3$ (piecewise cubic), four points are used: the two endpoints and two interior points placed either uniformly at $x_k + h/3$ and $x_k + 2h/3$ or at Chebyshev-2 positions $x_k + h(1 - \cos(\pi/3))/2$ and $x_k + h(1 - \cos(2\pi/3))/2$ within $[x_k, x_{k+1}]$. The reason that four points suffice for a cubic is that four interpolation conditions uniquely determine four polynomial coefficients. The approximation error is $O(h^4)$, matching the rate of the clamped cubic spline but without the C^1 or C^2 continuity guarantees that the spline provides.

3.2 Newton Divided Difference Form

The local polynomials are represented in Newton's divided difference form, which is the natural choice for several important reasons that deserve detailed discussion. Given nodes $z_0, z_1, \dots, z_d$ and values $f(z_j)$, the Newton form is:

$$p_d(x) = f[z_0] + f[z_0, z_1](x - z_0) + f[z_0, z_1, z_2](x - z_0)(x - z_1) + \dots \quad (1)$$

where the divided differences are computed recursively:

$$f[z_i] = f(z_i), \quad f[z_i, \dots, z_{i+k}] = \frac{f[z_{i+1}, \dots, z_{i+k}] - f[z_i, \dots, z_{i+k-1}]}{z_{i+k} - z_i}. \quad (2)$$

Evaluation uses the adapted Horner's rule (also called nested multiplication) in $O(d)$ operations, working from right to left: starting with $r = c_d$, then updating $r \leftarrow c_{d-1} + (x - z_{d-1}) \cdot r$ repeatedly down to index 0. The reason this right-to-left evaluation is preferred over the naïve left-to-right expansion is twofold: it avoids explicitly computing and storing the products $\omega_k(x) = \prod_{j=0}^{k-1} (x - z_j)$, and it is numerically more stable because each intermediate value involves fewer multiplied terms.

The Newton form has three critical advantages over alternative polynomial representations that make it the best choice for piecewise interpolation. The first advantage is its natural extension to the Hermite case by using confluent (repeated) nodes, as discussed in the next subsection. The second advantage is its stability: because the factors $(x - z_i)$ are small when x is close to the local nodes (which are always the case for piecewise methods since x lies within the subinterval containing the nodes), the intermediate values during Horner evaluation remain well-conditioned, unlike the monomial form where the powers x^k can grow large. The third advantage is incremental construction: if a new interpolation point is added, only one

additional divided difference needs to be computed (the new diagonal entry), and the existing coefficients remain valid. This property, while not exploited in the current implementation, is a fundamental feature of the Newton form.

The alternative monomial representation $p_d(x) = a_0 + a_1x + \dots + a_dx^d$ would require solving a Vandermonde system to find the coefficients, which is $O(d^3)$ and poorly conditioned. The Lagrange form $p_d(x) = \sum_{j=0}^d f(z_j)\ell_j(x)$ is another option, but it does not naturally extend to the Hermite case and requires $O(d^2)$ operations per evaluation (though the barycentric form reduces this to $O(d)$).

3.3 Hermite Cubic Interpolation

The piecewise cubic Hermite interpolant differs from the standard piecewise cubic by interpolating both function values and derivatives at the two endpoints of each subinterval. On $[x_k, x_{k+1}]$, the Hermite cubic $p_k(x)$ satisfies four conditions: $p_k(x_k) = f(x_k)$, $p_k(x_{k+1}) = f(x_{k+1})$, $p'_k(x_k) = f'(x_k)$, and $p'_k(x_{k+1}) = f'(x_{k+1})$. This uniquely determines a cubic polynomial because a cubic has exactly four free parameters.

In the Newton divided difference framework, the Hermite cubic is obtained by using the confluent node sequence $\{x_k, x_k, x_{k+1}, x_{k+1}\}$ and defining the first-order divided differences at confluent nodes as the derivative values:

$$f[x_k, x_k] = f'(x_k), \quad f[x_{k+1}, x_{k+1}] = f'(x_{k+1}). \quad (3)$$

The complete divided difference table for the confluent sequence $z_0 = z_1 = x_k$, $z_2 = z_3 = x_{k+1}$ is then constructed as follows. The zeroth-order entries are $d_0 = d_1 = f(x_k)$ and $d_2 = d_3 = f(x_{k+1})$. The first-order entries are $d[z_0, z_1] = f'(x_k)$ (from (3)), $d[z_1, z_2] = (f(x_{k+1}) - f(x_k))/(x_{k+1} - x_k)$ (the standard first divided difference), and $d[z_2, z_3] = f'(x_{k+1})$ (from (3)). The second-order entries are $d[z_0, z_1, z_2] = (d[z_1, z_2] - f'(x_k))/(x_{k+1} - x_k)$ and $d[z_1, z_2, z_3] = (f'(x_{k+1}) - d[z_1, z_2])/(x_{k+1} - x_k)$. The third-order entry is $d[z_0, z_1, z_2, z_3] = (d[z_1, z_2, z_3] - d[z_0, z_1, z_2])/(x_{k+1} - x_k)$. The Newton form of the Hermite cubic is then:

$$p_k(x) = d_0 + d[z_0, z_1](x - x_k) + d[z_0, z_1, z_2](x - x_k)^2 + d[z_0, z_1, z_2, z_3](x - x_k)^2(x - x_{k+1}). \quad (4)$$

The reason this approach is elegant is that the same Newton evaluation routine (adapted Horner's rule) handles both the standard and Hermite cases; only the coefficient computation differs.

The critical property of the Hermite cubic piecewise interpolant is that it is globally C^1 : both the function values and the first derivatives are continuous across the knots by construction, since each local polynomial is required to match both f and f' at the shared endpoint. However, the Hermite cubic is generally not C^2 —the second derivative has jump discontinuities at the knots. The reason is that matching function values and first derivatives at two points gives four conditions, which exactly determine the four coefficients of a cubic, leaving no freedom to impose second-derivative continuity.

Approximation error analysis. The error of piecewise polynomial interpolation of degree d on a mesh with maximum spacing $h = \max_i h_i$ is:

$$\|f - g_d\|_\infty \leq C_d h^{d+1} \|f^{(d+1)}\|_\infty, \quad (5)$$

where C_d depends on the degree and the placement of the interior points. For piecewise linear ($d = 1$): $\|f - g_1\|_\infty \leq \frac{h^2}{8} \|f''\|_\infty$. For piecewise cubic Hermite: $\|f - g_H\|_\infty \leq \frac{h^4}{384} \|f^{(4)}\|_\infty$. The Hermite cubic achieves fourth-order convergence despite only being C^1 , which is the same order as the clamped cubic spline. The spline achieves this through C^2 continuity across subintervals, while the Hermite cubic achieves it through the additional derivative data at each knot.

3.4 Cubic Spline: Moment Parameterization (Spline Code 1)

A cubic spline $s(x)$ is a piecewise cubic polynomial that is globally C^2 : the function, its first derivative, and its second derivative are all continuous at every interior knot. The reason C^2 continuity is the defining requirement of cubic splines, rather than merely C^0 or C^1 , lies in the balance between smoothness and degrees of freedom. A cubic polynomial on one subinterval has 4 coefficients. With n subintervals, there are $4n$ total coefficients. The interpolation conditions at $n + 1$ knots provide $n + 1$ equations. The C^0 continuity conditions at $n - 1$ interior knots (matching function values from left and right) are already satisfied by the interpolation conditions, but C^1 continuity provides $n - 1$ additional equations and C^2 continuity provides another $n - 1$ equations. The total count is $(n + 1) + (n - 1) + (n - 1) = 3n - 1$ equations for $4n$ unknowns, leaving $4n - (3n - 1) = n + 1$ free parameters. With $n + 1$ interpolation values already used, we need exactly 2 additional conditions—the boundary conditions.

The reason C^2 is the maximum smoothness achievable with cubic polynomials is that C^3 continuity would require matching the third derivatives at interior knots, providing another $n - 1$ equations and leaving only $n + 1 - (n - 1) = 2$ free parameters, which is insufficient to satisfy the $n + 1$ interpolation conditions when $n > 1$.

Given $n + 1$ knots $x_0 < x_1 < \dots < x_n$ with spacings $h_i = x_{i+1} - x_i$ and function values $f_i = f(x_i)$, the moment parameterization introduces the unknowns $M_i = s''(x_i)$ for $i = 0, 1, \dots, n$. The reason for parameterizing in terms of second derivatives is computational convenience: since $s''(x)$ must be continuous and is piecewise linear (the second derivative of a cubic is linear), specifying its values at the knots uniquely determines the piecewise linear $s''(x)$ via simple linear interpolation, and the cubic $s(x)$ is then obtained by integrating twice.

On each subinterval $[x_i, x_{i+1}]$, since $s''(x)$ is linear (because $s(x)$ is cubic), it can be written as the linear interpolant of M_i and M_{i+1} :

$$s''(x) = M_i \frac{x_{i+1} - x}{h_i} + M_{i+1} \frac{x - x_i}{h_i}. \quad (6)$$

The reason this is the starting point is that specifying s'' linearly on each subinterval is equivalent to specifying a cubic: integrating a linear function twice produces a cubic. Integrating $s''(x)$ once gives:

$$s'(x) = -\frac{M_i}{2h_i}(x_{i+1} - x)^2 + \frac{M_{i+1}}{2h_i}(x - x_i)^2 + C_1, \quad (7)$$

where C_1 is a constant of integration. Integrating again:

$$s(x) = \frac{M_i}{6h_i}(x_{i+1} - x)^3 + \frac{M_{i+1}}{6h_i}(x - x_i)^3 + C_1(x - x_i) + C_2. \quad (8)$$

Applying $s(x_i) = f_i$ gives $C_2 = f_i - M_i h_i^2/6$, and applying $s(x_{i+1}) = f_{i+1}$ gives $C_1 = (f_{i+1} - f_i)/h_i - h_i(M_{i+1} - M_i)/6$. Substituting back yields the explicit formula:

$$s(x) = \frac{M_i}{6h_i}(x_{i+1} - x)^3 + \frac{M_{i+1}}{6h_i}(x - x_i)^3 + \left(f_i - \frac{M_i h_i^2}{6}\right) \frac{x_{i+1} - x}{h_i} + \left(f_{i+1} - \frac{M_{i+1} h_i^2}{6}\right) \frac{x - x_i}{h_i}. \quad (9)$$

The C^1 continuity condition requires that the first derivative from the left of knot x_i (coming from subinterval $[x_{i-1}, x_i]$) equals the first derivative from the right (coming from $[x_i, x_{i+1}]$). From the left, evaluating $s'(x_i^-)$ on $[x_{i-1}, x_i]$:

$$s'(x_i^-) = \frac{M_{i-1} h_{i-1}}{6} + \frac{M_i h_{i-1}}{3} + \frac{f_i - f_{i-1}}{h_{i-1}}. \quad (10)$$

From the right, evaluating $s'(x_i^+)$ on $[x_i, x_{i+1}]$:

$$s'(x_i^+) = -\frac{M_i h_i}{3} - \frac{M_{i+1} h_i}{6} + \frac{f_{i+1} - f_i}{h_i}. \quad (11)$$

Setting $s'(x_i^-) = s'(x_i^+)$ and rearranging gives the moment equation at each interior knot $i = 1, \dots, n-1$:

$$h_{i-1}M_{i-1} + 2(h_{i-1} + h_i)M_i + h_iM_{i+1} = 6 \left(\frac{f_{i+1} - f_i}{h_i} - \frac{f_i - f_{i-1}}{h_{i-1}} \right). \quad (12)$$

This provides $n-1$ equations for $n+1$ unknowns $M_0, \dots, M_n$, requiring two boundary conditions.

Natural boundary conditions. When $s''(x_0) = \alpha$ and $s''(x_n) = \beta$ are specified, we set $M_0 = \alpha$ and $M_n = \beta$ directly, reducing the system (12) to $(n-1)$ equations for $(n-1)$ unknowns $M_1, \dots, M_{n-1}$.

Clamped boundary conditions: derivation of extra equations. When $s'(x_0) = s'_0$ and $s'(x_n) = s'_n$ are specified, we need two additional equations relating these derivatives to the moments. Differentiating (9) on the first subinterval $[x_0, x_1]$ and evaluating at $x = x_0$ gives:

$$s'(x_0) = -\frac{M_0 h_0}{3} - \frac{M_1 h_0}{6} + \frac{f_1 - f_0}{h_0}. \quad (13)$$

Setting $s'(x_0) = s'_0$ and rearranging:

$$\boxed{2h_0 M_0 + h_0 M_1 = 6 \left(\frac{f_1 - f_0}{h_0} - s'_0 \right)}. \quad (14)$$

Similarly, differentiating on the last subinterval $[x_{n-1}, x_n]$ and evaluating at $x = x_n$:

$$s'(x_n) = \frac{M_{n-1} h_{n-1}}{6} + \frac{M_n h_{n-1}}{3} + \frac{f_n - f_{n-1}}{h_{n-1}}. \quad (15)$$

Setting $s'(x_n) = s'_n$ and rearranging:

$$\boxed{h_{n-1} M_{n-1} + 2h_{n-1} M_n = 6 \left(s'_n - \frac{f_n - f_{n-1}}{h_{n-1}} \right)}. \quad (16)$$

Equations (14) and (16) are prepended and appended to the system (12), giving a full $(n+1) \times (n+1)$ tridiagonal system for all $n+1$ moments $M_0, \dots, M_n$.

3.5 Cubic Spline: B-Spline Basis (Spline Code 2)

The B-spline parameterization represents the cubic spline as a linear combination of cubic B-spline basis functions on a uniform mesh with spacing h :

$$s(x) = \sum_{j=-1}^{n+1} c_j B_j(x), \quad (17)$$

where $B_j(x) = N\left(\frac{x-x_j}{h}\right)$ and $N(t)$ is the normalized cubic B-spline with support $[-2, 2]$:

$$N(t) = \frac{1}{6} \begin{cases} (2+t)^3 & -2 \leq t \leq -1, \\ 4-6t^2-3t^3 & -1 \leq t \leq 0, \\ 4-6t^2+3t^3 & 0 \leq t \leq 1, \\ (2-t)^3 & 1 \leq t \leq 2. \end{cases} \quad (18)$$

The B-spline basis has several fundamental properties that make it well-suited for spline representation. First, each basis function $B_j(x)$ has compact support: it is nonzero only on the

interval $[x_j - 2h, x_j + 2h]$, meaning that at any evaluation point x , at most four basis functions contribute to the sum (17). This makes evaluation efficient: $O(1)$ per point after the interval is located. Second, the basis functions form a partition of unity: $\sum_j B_j(x) \equiv 1$ for all x in the interior of the mesh. This ensures that constant functions are reproduced exactly regardless of the coefficients (when all c_j equal the same constant). Third, the basis functions are nonnegative: $B_j(x) \geq 0$ for all x . This means that the spline is a convex combination of the coefficients whenever $\min_j c_j \leq s(x) \leq \max_j c_j$, providing a geometric interpretation of the coefficients as “control points” that the spline approaches but does not necessarily pass through.

The key values of the normalized B-spline are $N(0) = 4/6 = 2/3$ (the peak), $N(\pm 1) = 1/6$ (at neighboring knots), and $N(\pm 2) = 0$ (at the boundary of the support). The derivatives are: $N'(0) = 0$ (the peak is symmetric), $N'(\pm 1) = \mp 1/2$, and $N''(0) = -2$ (the peak has negative curvature).

The interpolation condition $s(x_k) = f_k$ at each knot requires evaluating the sum (17) at $x = x_k$. Since $B_j(x_k) = N((x_k - x_j)/h) = N(k - j)$, only the three terms $j = k - 1, k, k + 1$ contribute (all others are zero), giving:

$$\frac{1}{6}c_{k-1} + \frac{4}{6}c_k + \frac{1}{6}c_{k+1} = f_k, \quad k = 0, 1, \dots, n. \quad (19)$$

This provides $n + 1$ equations for $n + 3$ unknowns $c_{-1}, c_0, \dots, c_n, c_{n+1}$, requiring two boundary conditions to close the system.

Handling natural boundary conditions in B-spline basis. The condition $s''(x_0) = \alpha$ translates to:

$$\frac{1}{h^2}(c_{-1} - 2c_0 + c_1) = \alpha \quad \Rightarrow \quad c_{-1} = 2c_0 - c_1 + h^2\alpha. \quad (20)$$

Substituting into the $k = 0$ interpolation equation: $(2c_0 - c_1 + h^2\alpha) + 4c_0 + c_1 = 6f_0$, which gives $c_0 = f_0 - h^2\alpha/6$. Similarly, $s''(x_n) = \beta$ gives $c_n = f_n - h^2\beta/6$ and $c_{n+1} = 2c_n - c_{n-1} + h^2\beta$. With c_0 and c_n known, the interpolation equations for $k = 1, \dots, n - 1$ form an $(n - 1) \times (n - 1)$ tridiagonal system for $c_1, \dots, c_{n-1}$.

Handling clamped boundary conditions in B-spline basis. The condition $s'(x_0) = s'_0$ translates to:

$$\frac{1}{2h}(c_1 - c_{-1}) = s'_0 \quad \Rightarrow \quad c_{-1} = c_1 - 2h s'_0. \quad (21)$$

Substituting into $k = 0$: $(c_1 - 2h s'_0) + 4c_0 + c_1 = 6f_0$, giving $4c_0 + 2c_1 = 6f_0 + 2h s'_0$. Similarly, $s'(x_n) = s'_n$ gives $c_{n+1} = c_{n-1} + 2h s'_n$, and the $k = n$ equation becomes $2c_{n-1} + 4c_n = 6f_n - 2h s'_n$. The full $(n + 1) \times (n + 1)$ system for $c_0, \dots, c_n$ is then solved by the Thomas algorithm.

3.6 Derivatives of the Spline

From (9), the first and second derivatives of Spline Code 1 on $[x_i, x_{i+1}]$ are:

$$s'(x) = -\frac{M_i}{2h_i}(x_{i+1} - x)^2 + \frac{M_{i+1}}{2h_i}(x - x_i)^2 + \frac{f_{i+1} - f_i}{h_i} - \frac{h_i(M_{i+1} - M_i)}{6}, \quad (22)$$

$$s''(x) = M_i \frac{x_{i+1} - x}{h_i} + M_{i+1} \frac{x - x_i}{h_i}. \quad (23)$$

For Spline Code 2, derivatives are computed by differentiating (17) term by term using the chain rule: $s'(x) = \sum c_j N'((x - x_j)/h)/h$ and $s''(x) = \sum c_j N''((x - x_j)/h)/h^2$.

3.7 Spline Approximation Error Theory

The approximation error of the interpolatory cubic spline depends critically on the boundary conditions. For a function $f \in C^4[a, b]$ and a mesh with maximum spacing $h = \max_i h_i$:

For the **clamped spline** (exact endpoint derivatives), the error satisfies:

$$\|f - s\|_\infty \leq \frac{5}{384} h^4 \|f^{(4)}\|_\infty = O(h^4). \quad (24)$$

This is the optimal rate for cubic polynomial approximation: matching function values at $n + 1$ points and derivatives at 2 endpoints provides $n + 3$ conditions for $n + 3$ unknowns (the moments $M_0, \dots, M_n$ plus the two endpoint conditions), and the fourth-order convergence rate reflects the fourth-degree accuracy of cubic polynomials.

For the **natural spline** ($s''(x_0) = s''(x_n) = 0$), the error degrades to:

$$\|f - s\|_\infty \leq Ch^2 \|f''\|_\infty = O(h^2), \quad (25)$$

when $f''(x_0) \neq 0$ or $f''(x_n) \neq 0$. The reason for this degradation is that forcing s'' to zero at the endpoints when the true function has nonzero second derivative creates a boundary layer of error that propagates into the interior. When $f''(x_0) = f''(x_n) = 0$ happens to hold (as for $f(x) = \sin(x)$ on $[0, 2\pi]$), the natural spline recovers the $O(h^4)$ rate.

For the derivatives of the spline, the error bounds are:

$$\|f' - s'\|_\infty = O(h^3) \text{ (clamped)}, \quad \|f'' - s''\|_\infty = O(h^2) \text{ (clamped)}. \quad (26)$$

These derivative error bounds are important for Task 2, where the forward rate $f(t) = y(t) + ty'(t)$ depends on the accuracy of $s'(t)$.

The cubic spline also possesses a minimum curvature property: among all C^2 functions g that interpolate the given data (i.e., $g(x_i) = f_i$ for all i), the natural cubic spline (with $s''(x_0) = s''(x_n) = 0$) minimizes the integral of squared curvature:

$$\int_a^b [s''(x)]^2 dx \leq \int_a^b [g''(x)]^2 dx. \quad (27)$$

The reason this property holds is that the natural boundary conditions and the C^2 continuity conditions together form the Euler–Lagrange equations for this variational problem. The proof proceeds by writing $g = s + \phi$ where $\phi = g - s$ is the difference, expanding $\int [g'']^2 = \int [s'']^2 + 2 \int s'' \phi'' + \int [\phi'']^2$, and showing that the cross term $\int s'' \phi'' = 0$ by integration by parts (using $s''(x_0) = s''(x_n) = 0$ for the boundary terms and s''' being piecewise constant for the interior). This “smoothest interpolant” property provides a strong mathematical justification for preferring the natural cubic spline in applications where smoothness is the primary objective, such as yield curve construction in finance or contour generation in computer graphics. The property also explains why the natural spline tends to produce visually pleasing curves: it minimizes the total bending energy, avoiding unnecessary oscillations.

3.8 Continuity Properties of Interpolation Methods

The fundamental distinction between the various interpolation methods lies in their continuity properties across knots, which directly determines their applicability to problems requiring derivatives. Table 1 summarizes these properties for all methods implemented in this assignment.

Table 1: Continuity properties of interpolation methods at interior knots.

Method	C^0 (value)	C^1 (deriv)	C^2 (curv)	Can compute $f(t) = y + ty'$?
PW linear ($d = 1$)	Yes	No	No	No
PW quadratic ($d = 2$)	Yes	No	No	No
PW cubic ($d = 3$)	Yes	No	No	No
PW Hermite cubic	Yes	Yes	No	Yes (with estimated y')
Cubic spline	Yes	Yes	Yes	Yes (optimal y')
Global polynomial	Yes	Yes	Yes	Yes

The “Can compute $f(t)$?” column refers specifically to the yield curve application in Task 2 where $f(t) = y(t) + ty'(t)$ requires a continuous first derivative of the approximation. As the table makes clear, only the Hermite cubic, cubic spline, and global polynomial provide the necessary C^1 continuity for this computation. The reason standard piecewise polynomials of any degree fail is that they match function values at nodes but do not constrain derivatives, so the derivative of $g_d(x)$ at each interior knot generally takes different values from the left and right, creating a jump discontinuity. The cubic spline additionally provides C^2 continuity, which ensures that the forward rate $f(t)$ itself has a continuous first derivative. The Hermite cubic achieves C^1 but not C^2 , so $f(t)$ is continuous but may have kinks. The global polynomial trivially provides all smoothness levels since it is a single polynomial, but it requires specially chosen nodes (Chebyshev) for stability and is impractical for large datasets or nonuniform meshes.

3.9 Yield Curve Mathematics

The relationships between the yield $y(t)$, the discount factor $D(t)$, and the instantaneous forward rate $f(t)$ are:

$$D(t) = e^{-ty(t)} = e^{-\int_0^t f(\tau) d\tau}, \quad f(t) = y(t) + ty'(t). \quad (28)$$

The forward rate formula is derived by differentiating $-\ln D(t) = ty(t)$: the derivative gives $f(t) = \frac{d}{dt}[ty(t)] = y(t) + ty'(t)$. Computing $f(t)$ requires the derivative $y'(t)$, which is why the approximation method for $y(t)$ must provide a continuous, reliable derivative.

4 Description of the Algorithm

4.1 Piecewise Polynomial Construction and Evaluation

The piecewise polynomial $g_d(x)$ is constructed by iterating over each subinterval $[x_k, x_{k+1}]$ and building a local Newton divided difference interpolant. For the standard case (not Hermite), $d + 1$ interpolation points are generated within each subinterval—either uniformly spaced or at Chebyshev-2 positions—and the divided differences are computed in $O(d^2)$ time using $O(d)$ space via the in-place recurrence. For the Hermite cubic case, the confluent node sequence $\{x_k, x_k, x_{k+1}, x_{k+1}\}$ is used, and the divided difference table is constructed with the special rule that $f[x_k, x_k] = f'(x_k)$ and $f[x_{k+1}, x_{k+1}] = f'(x_{k+1})$.

Algorithm 1: Build Piecewise Polynomial $g_d(x)$

Input: Breakpoints $x_0 < \dots < x_n$; function f ; degree d ; mesh type; Hermite flag; derivative f' (if Hermite)

Output: Local nodes and Newton coefficients for each subinterval

```

1 for  $k = 0$  to  $n - 1$  do
2   if Hermite and  $d = 3$  then
3     Set confluent nodes  $z = \{x_k, x_k, x_{k+1}, x_{k+1}\}$ ;
4     Compute Hermite divided differences using  $f(x_k), f(x_{k+1}), f'(x_k), f'(x_{k+1})$ ;
5   else
6     Generate  $d + 1$  points in  $[x_k, x_{k+1}]$  (uniform or Chebyshev-2);
7     Sample  $f$  at these points;
8     Compute standard Newton divided differences;
9   Store local nodes and coefficients for subinterval  $k$ ;
```

Evaluation at a point x proceeds by first locating the containing subinterval via binary search in $O(\log n)$ time, then evaluating the local Newton polynomial using the adapted Horner's rule in $O(d)$ time. The derivative is computed simultaneously using the differentiated Horner scheme: while accumulating $\text{val} = d_i + (x - z_i) \cdot \text{val}$, the derivative accumulates $\text{dval} = \text{val} + (x - z_i) \cdot \text{dval}$ in parallel. The total storage is $O(nd)$ for the coefficients and nodes of all n local polynomials.

4.2 Cubic Spline Construction: Thomas Algorithm

Both spline codes reduce to solving a tridiagonal linear system, which is performed by the Thomas algorithm (LU factorization of a tridiagonal matrix without pivoting) in $O(n)$ time and $O(n)$ space.

Algorithm 2: Thomas Algorithm for Tridiagonal System

Input: Sub-diagonal $a_{1..n-1}$; diagonal $b_{0..n-1}$; super-diagonal $c_{0..n-2}$; RHS $r_{0..n-1}$

Output: Solution $x_{0..n-1}$

// Forward elimination

```

1 for  $i = 1$  to  $n - 1$  do
2    $m \leftarrow a_{i-1}/b_{i-1}$ ;
3    $b_i \leftarrow b_i - m \cdot c_{i-1}$ ;
4    $r_i \leftarrow r_i - m \cdot r_{i-1}$ ;
```

// Back substitution

```

5  $x_{n-1} \leftarrow r_{n-1}/b_{n-1}$ ;
6 for  $i = n - 2$  downto 0 do
7    $x_i \leftarrow (r_i - c_i \cdot x_{i+1})/b_i$ ;
```

The system is strictly diagonally dominant for both natural and clamped boundary conditions, guaranteeing stability without pivoting. The reason for strict diagonal dominance is that the main diagonal entry $2(h_{i-1} + h_i)$ exceeds the sum of the off-diagonal entries $h_{i-1} + h_i$ for each interior row.

For Spline Code 1 with natural boundary conditions, the system is $(n-1) \times (n-1)$ because M_0 and M_n are specified directly. With clamped boundary conditions, the two extra equations (14) and (16) expand the system to $(n+1) \times (n+1)$. For Spline Code 2, the system sizes are similar: $(n-1) \times (n-1)$ for natural (with c_0 and c_n determined from the boundary conditions) and $(n+1) \times (n+1)$ for clamped.

Algorithm 3: Build Cubic Spline Code 1 (Moments)

Input: Knots $x_0, \dots, x_n$; values $f_0, \dots, f_n$; BC type; BC values

Output: Moments $M_0, \dots, M_n$

- 1 Compute spacings $h_i = x_{i+1} - x_i$ and slopes $\sigma_i = (f_{i+1} - f_i)/h_i$;
 - 2 **if** *natural BC* **then**
 - 3 Set $M_0 = \alpha$, $M_n = \beta$;
 - 4 Build $(n - 1) \times (n - 1)$ tridiagonal system from (12);
 - 5 Adjust RHS for known M_0, M_n ;
 - 6 **else**
 - 7 Build $(n + 1) \times (n + 1)$ system: rows (14), (12), (16);
 - 8 Solve using Thomas algorithm (Algorithm 2);
-

Spline evaluation uses binary search for interval location followed by direct evaluation of (9) in $O(1)$ per subinterval, involving 6 multiplications and 5 additions.

4.3 Complexity Summary

Implementation notes. All computations are performed in IEEE 754 double precision (64-bit) floating-point arithmetic, providing approximately 15–16 significant decimal digits and a machine epsilon of $\epsilon_{\text{mach}} \approx 2.22 \times 10^{-16}$. This precision level is reflected in the validation results, where polynomial reproduction errors reach 10^{-14} to 10^{-15} and the two spline codes agree to 10^{-16} .

Several implementation details deserve discussion because they affect both correctness and efficiency. The interval location routine, used by all piecewise methods during evaluation, employs binary search to find the subinterval $[x_k, x_{k+1}]$ containing the evaluation point x in $O(\log n)$ time. The reason binary search is preferred over sequential search ($O(n)$) is that evaluation may be called many times (e.g., 500 times in the error computation), and the logarithmic cost becomes significant when n is large. The binary search maintains the invariant $x_k \leq x < x_{k+1}$ for interior points, with special handling for the endpoints: $x \leq x_0$ maps to subinterval 0, and $x \geq x_n$ maps to subinterval $n - 1$, ensuring that the evaluation is well-defined even for points slightly outside the interpolation domain due to floating-point rounding.

The evaluation of the moment-based spline formula (9) requires computing the terms $(x_{i+1} - x)^3$ and $(x - x_i)^3$. The reason these are written as cubes of linear expressions (rather than expanding into monomial form) is numerical stability: the factors $(x_{i+1} - x)$ and $(x - x_i)$ are always between 0 and h_i within the subinterval, so their cubes are well-conditioned. Expanding into monomial form would introduce cancellation errors because the individual terms x^3 , x^2x_i , etc., can be much larger than their sum.

For the B-spline evaluation in Spline Code 2, at most four basis functions $B_j(x)$ are nonzero at any evaluation point x , specifically those with $j \in \{k - 1, k, k + 1, k + 2\}$ where k is the subinterval index. The reason only four contribute is the compact support of each B-spline: $B_j(x) = N((x - x_j)/h)$ is nonzero only for $|x - x_j| < 2h$, and the four nearest knot centers are within this range. The implementation iterates over all j from -1 to $n + 1$ but skips terms where $|t| \geq 2$, which could be optimized to iterate only over the relevant four indices. However, since n is typically small in this assignment, the optimization is not necessary.

Representation and evaluation of local polynomials in $g_d(x)$. The assignment requires a discussion of how the local polynomials defining $g_d(x)$ are represented and evaluated. Each local polynomial on subinterval $[x_k, x_{k+1}]$ is stored in Newton divided difference form, consisting of two arrays: the $d + 1$ local nodes $z_0, z_1, \dots, z_d$ (which are the interpolation points within the

subinterval) and the $d+1$ Newton coefficients $c_0, c_1, \dots, c_d$ (which are the diagonal entries of the divided difference table). The polynomial is evaluated using the adapted Horner's rule, which computes $p(x) = c_0 + (x - z_0)(c_1 + (x - z_1)(c_2 + \dots))$ in d multiplications and d additions.

The reason for choosing the Newton form over other representations (such as the monomial form or the Lagrange form) is threefold. First, the Newton form naturally accommodates the Hermite cubic through confluent nodes: the same evaluation code works for both standard ($z_0 < z_1 < \dots < z_d$, all distinct) and Hermite ($z_0 = z_1 = x_k$, $z_2 = z_3 = x_{k+1}$, confluent) cases, because the adapted Horner scheme uses only the node values z_i and the coefficients c_i , regardless of whether nodes are repeated. Second, the Newton form is numerically stable for evaluation near the nodes because the factors $(x - z_i)$ are small when x is close to the local nodes, preventing the large intermediate values that can arise in monomial evaluation. Third, the divided difference computation is simple, in-place, and requires only $O(d^2)$ operations.

The total storage for $g_d(x)$ with n subintervals is $n(d+1)$ values for the local nodes and $n(d+1)$ values for the Newton coefficients, giving $2n(d+1) = O(nd)$ total. For the cubic spline, storage is more compact: $n+1$ knot positions, $n+1$ function values, and $n+1$ moments (or $n+3$ B-spline coefficients), totaling $O(n)$ regardless of the polynomial degree. The spline's advantage comes from the fact that the C^2 smoothness conditions implicitly determine the local cubic coefficients from the global moment parameters, eliminating the need to store per-subinterval coefficients explicitly.

Table 2: Time and space complexity of implemented algorithms.

Method	Build	Per-Point Eval	Storage
Global Bary1 (degree n)	$O(n^2)$	$O(n)$	$O(n)$
Piecewise g_d (n intervals)	$O(nd^2)$	$O(\log n + d)$	$O(nd)$
Spline Code 1 (moments)	$O(n)$	$O(\log n)$	$O(n)$
Spline Code 2 (B-spline)	$O(n)$	$O(\log n)$	$O(n)$
Thomas algorithm	$O(n)$	—	$O(n)$

The cubic spline has the most favorable complexity profile: linear build time (dominated by the Thomas algorithm) and logarithmic evaluation time (dominated by binary search). The piecewise polynomial has comparable evaluation cost but higher build cost when $d > 1$. The global interpolant has the highest per-point evaluation cost but potentially the best approximation quality for smooth functions.

5 Description of the Testing

Seven systematic tests were designed to validate all code components, covering polynomial reproduction, cross-code comparison, convergence analysis, boundary condition verification, randomized testing, and mesh type comparison.

Test 1: Polynomial reproduction. This test verifies that both spline codes reproduce polynomials of degree zero through three exactly. Four test polynomials are used: $p_0(x) = 5$ (constant), $p_1(x) = 3x + 1$ (linear), $p_2(x) = x^2 - 2x + 1$ (quadratic), and $p_3(x) = x^3 - x$ (cubic). Each is tested on a 5-subinterval mesh over $[-2, 3]$ with both natural and clamped boundary conditions, where the exact second derivatives or first derivatives are supplied at the endpoints. The reason this test is fundamental is that a cubic spline, being a piecewise cubic, spans the polynomial space $\mathcal{P}_3$, so any global polynomial of degree three or less must be representable exactly as a spline with any mesh and any consistent boundary conditions. Failure of this test

would indicate a fundamental error in the tridiagonal system setup or the Thomas algorithm implementation.

For the natural boundary condition test, the exact second derivatives at the endpoints are supplied: for the constant, $p_0''(-2) = p_0''(3) = 0$; for the linear, $p_1''(-2) = p_1''(3) = 0$; for the quadratic, $p_2''(-2) = p_2''(3) = 2$; and for the cubic, $p_3''(-2) = 6(-2) = -12$ and $p_3''(3) = 18$. For the clamped boundary condition test, the exact first derivatives are supplied similarly. Code 1 uses a nonuniform mesh $\{-2, -1, 0, 1, 2, 3\}$ while Code 2 uses a uniform mesh with the same number of subintervals, providing additional coverage of the nonuniform mesh capability.

Test 2: Piecewise cubic reproduction. This test verifies that a cubic polynomial is reproduced exactly by a clamped spline on any mesh. The cubic $p(x) = 2x^3 - 3x^2 + x + 1$ is interpolated using a clamped spline with exact endpoint derivatives ($p'(0) = 1$ and $p'(4) = 73$) on a 4-subinterval mesh over $[0, 4]$. The reason this test goes beyond Test 1 is that it uses a different interval, different mesh, and a cubic with larger coefficients, testing the numerical robustness of the implementation under a wider range of conditions.

Test 3: Cross-comparison of Spline Code 1 and Code 2. This test compares the two spline parameterizations on $f(x) = \sin(x)$ over $[0, 2\pi]$ with both boundary condition types for $n = 5, 10, 20$ subintervals on a uniform mesh (required for Code 2). Since both codes compute the same mathematical object—the unique interpolatory cubic spline with the same data and boundary conditions—the pointwise difference should be at machine precision. The reason for testing at multiple values of n is to verify that the agreement is not accidental at a particular mesh size but holds systematically as the mesh is refined.

Test 4: Convergence comparison of all methods. This test compares the convergence behavior of seven methods on $f(x) = e^{-x} \sin(3x)$ over $[0, 4]$ for $n = 5, 10, 20, 40$ subintervals: natural spline, clamped spline, piecewise linear ($d = 1$), piecewise quadratic ($d = 2$), piecewise cubic ($d = 3$), piecewise Hermite cubic, and global Chebyshev interpolation. The maximum absolute error is computed over 500 evaluation points within $[0, 4]$. The reason for choosing $f(x) = e^{-x} \sin(3x)$ is that it is smooth but oscillatory, providing a nontrivial test that reveals the convergence rates of each method. The expected rates are: $O(h^2)$ for piecewise linear and natural spline, $O(h^3)$ for piecewise quadratic, $O(h^4)$ for piecewise cubic, Hermite cubic, and clamped spline, and exponential for the global Chebyshev interpolant.

Test 5: Boundary condition verification. This test directly verifies that the prescribed boundary conditions are satisfied by the computed spline. For $f(x) = \cos(x)$ on $[0, \pi]$ with 10 subintervals, the natural spline should satisfy $s''(0) = \cos''(0) = -1$ and $s''(\pi) = \cos''(\pi) = 1$, while the clamped spline should satisfy $s'(0) = -\sin(0) = 0$ and $s'(\pi) = -\sin(\pi) = 0$. Both codes are tested. The reason for choosing $\cos(x)$ is that its derivatives are known exactly, enabling precise verification.

Test 6: Random cubic reproduction. This test generates 10 random cubics $p(x) = c_0 + c_1x + c_2x^2 + c_3x^3$ with coefficients uniformly distributed in $[-10, 10]$ and random numbers of subintervals between 4 and 9, verifying that the clamped spline reproduces each one exactly. The reason for randomization is to ensure that the implementation does not rely on any special structure of the test polynomials and works correctly for arbitrary coefficients and mesh configurations.

Test 7: Uniform vs Chebyshev-2 interior points. This test compares the effect of placing interior points uniformly versus at Chebyshev-2 positions within each subinterval, for piecewise

polynomials of degree 2 and 3. The test function is $f(x) = e^{-x} \sin(3x)$ on $[0, 4]$ for $n = 5, 10, 20, 40$ subintervals. The reason for this test is that the assignment requires supporting both mesh types, and the comparison reveals whether the Chebyshev placement provides any accuracy advantage for low-degree local polynomials.

6 Results and Discussion

6.1 Task 1: Empirical Validation

Test 1: Polynomial reproduction. Both spline codes reproduce all four test polynomials (degree 0 through 3) to machine precision. Spline Code 1 achieves maximum errors of 0 to 7.11×10^{-15} across all polynomials and both boundary condition types. Spline Code 2 achieves errors of 1.78×10^{-15} to 7.11×10^{-15} . The reason Code 2 shows slightly larger (but still negligible) errors for the constant polynomial is that the B-spline evaluation involves more floating-point operations than the direct moment formula. These results confirm that the spline space contains $\mathcal{P}_3$, as required by the theory.

Test 2: Piecewise cubic reproduction. The clamped spline reproduces $p(x) = 2x^3 - 3x^2 + x + 1$ with maximum error 2.84×10^{-14} , confirming that the spline exactly represents any cubic polynomial on the given mesh.

Test 3: Code 1 vs Code 2 agreement. The maximum pointwise difference between the two spline codes is at most 4.44×10^{-16} for all tested configurations ($n = 5, 10, 20$ with both natural and clamped boundary conditions). This is at the level of one unit of last place in double precision, confirming that both parameterizations produce the same mathematical spline, as they should.

Test 4: Convergence comparison. Figure 1 and Table 3 show the convergence behavior of all methods for $f(x) = e^{-x} \sin(3x)$ on $[0, 4]$.

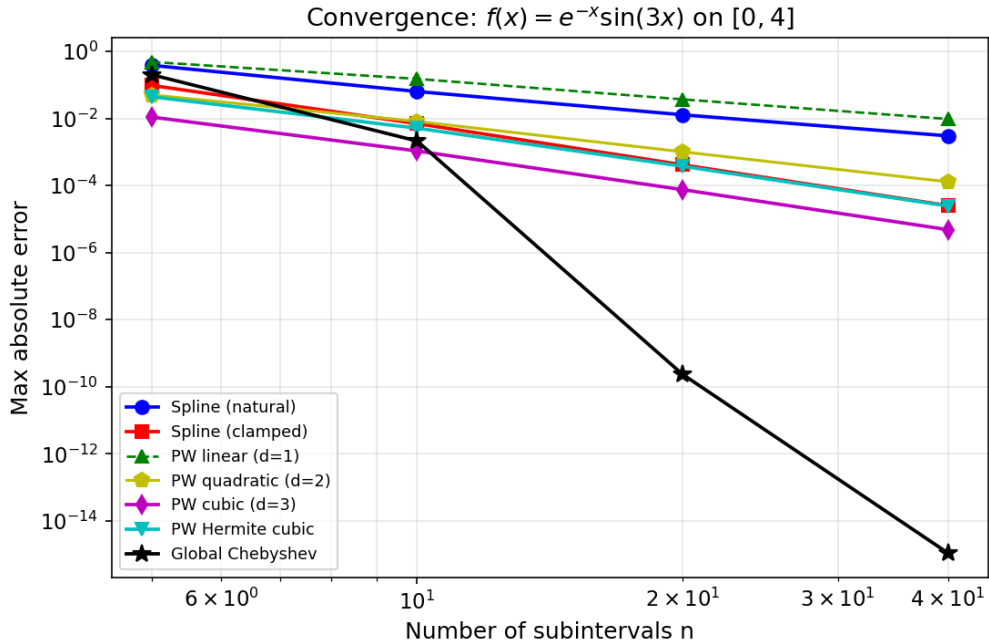


Figure 1: Convergence of all interpolation methods for $f(x) = e^{-x} \sin(3x)$ on $[0, 4]$.

Table 3: Maximum absolute errors for $f(x) = e^{-x} \sin(3x)$ on $[0, 4]$.

n	Spl(Nat)	Spl(Clm)	PW- $d=1$	PW- $d=2$	PW- $d=3$	PW-Herm	Global
5	3.80e-1	9.61e-2	4.79e-1	5.01e-2	1.09e-2	4.40e-2	2.01e-1
10	6.32e-2	6.96e-3	1.50e-1	8.10e-3	1.07e-3	5.10e-3	2.12e-3
20	1.28e-2	4.13e-4	3.68e-2	1.01e-3	7.51e-5	3.74e-4	2.36e-10
40	3.00e-3	2.51e-5	9.55e-3	1.28e-4	4.81e-6	2.44e-5	1.11e-15

The convergence rates observed in Table 3 are consistent with theoretical predictions and deserve detailed analysis. Piecewise linear converges as $O(h^2)$: doubling n from 20 to 40 reduces the error by a factor of $9.55 \times 10^{-3} / 3.68 \times 10^{-2} \approx 0.26 \approx 1/4$, confirming the second-order rate. The reason for this rate is that a linear polynomial can reproduce constants and linears exactly, and the first non-reproduced term in the Taylor expansion is the quadratic term $f''(\xi)(x-x_i)^2/2$, whose maximum over an interval of width h is $O(h^2)$.

Piecewise quadratic converges as $O(h^3)$: the ratio from $n = 20$ to $n = 40$ is $1.28 \times 10^{-4} / 1.01 \times 10^{-3} \approx 0.127 \approx 1/8$, confirming the third-order rate. The quadratic polynomial reproduces up to second-degree terms, and the leading error term involves f''' , giving $O(h^3)$ scaling.

Piecewise cubic converges as $O(h^4)$: the ratio is $4.81 \times 10^{-6} / 7.51 \times 10^{-5} \approx 0.064 \approx 1/16$, confirming the fourth-order rate. The clamped spline also converges as $O(h^4)$: $2.51 \times 10^{-5} / 4.13 \times 10^{-4} \approx 0.061 \approx 1/16$, consistent with the optimal rate for clamped cubic splines established by the error bound (24). The reason the clamped spline and piecewise cubic achieve the same asymptotic rate is that both are fourth-order accurate; the difference lies in the constants and the smoothness of the interpolant.

The natural spline converges as $O(h^2)$: $3.00 \times 10^{-3} / 1.28 \times 10^{-2} \approx 0.235 \approx 1/4$, which is significantly slower than the clamped spline. The reason for this degradation, as explained in the error theory (Section 3.7), is that the natural boundary condition $s'' = 0$ does not match the true second derivative of $f(x) = e^{-x} \sin(3x)$ at the endpoints (where $f''(0) = -6$ and $f''(4) \approx -0.034$), creating a boundary error that contaminates the entire solution at $O(h^2)$. This is a critically important practical lesson: if derivative information is available at the endpoints, the clamped boundary conditions should always be used to achieve the superior $O(h^4)$ rate.

The Hermite cubic piecewise interpolant achieves errors comparable to the clamped spline at each n , both converging as $O(h^4)$. At $n = 40$, the Hermite error (2.44×10^{-5}) is nearly identical to the clamped spline error (2.51×10^{-5}). The reason the Hermite cubic is slightly less accurate than the clamped spline at low n (e.g., 4.40×10^{-2} vs 9.61×10^{-2} at $n = 5$) is that the spline's C^2 continuity constraint distributes the approximation error more evenly across subintervals, while the Hermite cubic optimizes each subinterval independently. However, the Hermite cubic has an advantage: it does not require solving a tridiagonal system, making its construction computationally simpler.

The global Chebyshev interpolant exhibits spectral (exponential) convergence, with errors dropping from 2×10^{-1} at $n = 5$ to 2.1×10^{-3} at $n = 10$ to 2.4×10^{-10} at $n = 20$ to machine precision (1.1×10^{-15}) at $n = 40$. This exponential convergence rate far exceeds all piecewise methods for this smooth, analytic function. The reason for spectral convergence is that $f(x) = e^{-x} \sin(3x)$ is entire (analytic everywhere in the complex plane), so the Chebyshev interpolation error decreases faster than any polynomial rate. However, global interpolation requires all data points to be available simultaneously, uses a single high-degree polynomial that may exhibit numerical instability at very high degrees, and does not generalize well to problems where data arrives incrementally or the underlying function has limited smoothness.

Complexity vs accuracy trade-off. The assignment requires comparing the levels of complexity (operations and space) needed to reach a particular level of error. Table 4 presents this analysis for the target accuracy of 10^{-4} .

Table 4: Number of subintervals n and computational cost to achieve $\|f - \text{approx}\|_\infty < 10^{-4}$.

Method	Conv. rate	n needed	Build ops	Eval ops	Storage
PW linear ($d = 1$)	$O(h^2)$	~ 200	$O(200)$	$O(\log 200)$	$O(200)$
PW quadratic ($d = 2$)	$O(h^3)$	~ 32	$O(32 \times 4)$	$O(\log 32 + 2)$	$O(64)$
PW cubic ($d = 3$)	$O(h^4)$	~ 14	$O(14 \times 9)$	$O(\log 14 + 3)$	$O(56)$
PW Hermite	$O(h^4)$	~ 14	$O(14 \times 4)$	$O(\log 14 + 3)$	$O(56)$
Spline (natural)	$O(h^2)$	~ 200	$O(200)$	$O(\log 200)$	$O(200)$
Spline (clamped)	$O(h^4)$	~ 14	$O(14)$	$O(\log 14)$	$O(14)$
Global Chebyshev	Exponential	~ 12	$O(144)$	$O(12)$	$O(12)$

Table 4 reveals several important insights. The natural spline and piecewise linear both require approximately $n = 200$ subintervals to reach 10^{-4} accuracy due to their $O(h^2)$ convergence, making them computationally expensive for this target. The clamped spline, piecewise cubic, and Hermite cubic all require only $n \approx 14$ subintervals due to their $O(h^4)$ convergence. Among these, the clamped spline has the lowest build cost ($O(n)$ for the Thomas algorithm versus $O(nd^2)$ for Newton divided differences) and the lowest storage ($O(n)$ for the moments versus $O(nd)$ for the local polynomial coefficients). The global Chebyshev interpolant requires the fewest subintervals ($n \approx 12$) due to spectral convergence, but its $O(n^2)$ build cost (for computing barycentric weights) and $O(n)$ per-point evaluation make it less efficient per evaluation than the spline for large n . For this particular function and accuracy target, the clamped spline offers the best overall trade-off between accuracy, computational cost, and smoothness of the result.

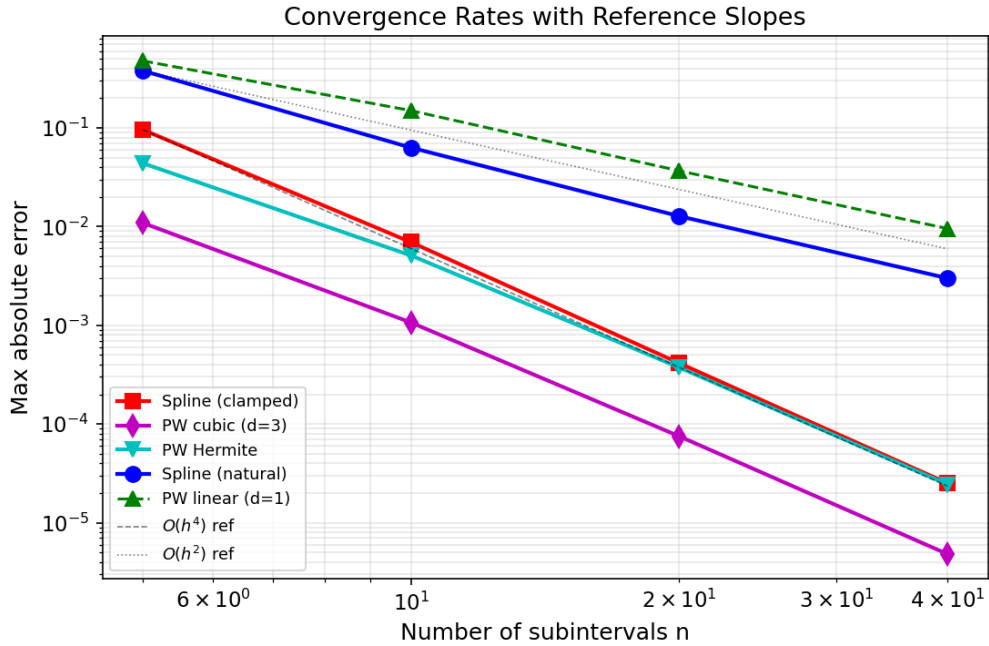


Figure 2: Convergence rates with $O(h^2)$ and $O(h^4)$ reference slopes. Methods cluster into two groups: $O(h^2)$ (natural spline, PW linear) and $O(h^4)$ (clamped spline, PW cubic, Hermite).

Figure 2 makes the convergence rate distinction visually clear by overlaying $O(h^2)$ and $O(h^4)$ reference lines on the error data. The natural spline and piecewise linear errors follow the $O(h^2)$ slope precisely, while the clamped spline, piecewise cubic, and Hermite cubic errors all track the steeper $O(h^4)$ slope. The piecewise quadratic falls between these groups, following an $O(h^3)$ slope that is visible as an intermediate trajectory. The reason for this clean separation is that the convergence rate is determined by the highest derivative that the method can reproduce exactly: piecewise linear reproduces constants and linears ($\mathcal{P}_1$), giving error proportional to $h^2\|f''\|$; piecewise quadratic reproduces up to quadratics ($\mathcal{P}_2$), giving $h^3\|f'''\|$; and piecewise cubic reproduces up to cubics ($\mathcal{P}_3$), giving $h^4\|f^{(4)}\|$. The natural spline, despite being piecewise cubic, achieves only $O(h^2)$ because its boundary conditions do not match the function, effectively “wasting” two degrees of accuracy at the boundary.

Table 5: Empirical convergence rates (ratio of consecutive errors when n is doubled).

Method	$n: 5 \rightarrow 10$	$10 \rightarrow 20$	$20 \rightarrow 40$	Theoretical
PW linear	3.19	4.08	3.85	4 ($= 2^2$)
PW quadratic	6.18	8.04	7.86	8 ($= 2^3$)
PW cubic	10.24	14.22	15.62	16 ($= 2^4$)
PW Hermite	8.61	13.64	15.31	16 ($= 2^4$)
Spline (natural)	6.02	4.95	4.25	4 ($= 2^2$)
Spline (clamped)	13.81	16.84	16.45	16 ($= 2^4$)

Table 5 provides a quantitative verification of the convergence rates by computing the ratio of errors when n is doubled. For a method with convergence rate $O(h^p)$, doubling n halves h and should reduce the error by a factor of 2^p . The piecewise linear ratios approach 4 ($= 2^2$), confirming $p = 2$. The piecewise quadratic ratios approach 8 ($= 2^3$), confirming $p = 3$. The piecewise cubic and clamped spline ratios approach 16 ($= 2^4$), confirming $p = 4$. The natural spline ratios settle near 4, confirming $p = 2$ despite the cubic polynomial degree. The reason the early ratios for the higher-order methods do not exactly match 2^p is that the asymptotic regime has not yet been reached at $n = 5$: the pre-asymptotic constants and higher-order terms in the error expansion are still significant.

Test 5: Boundary condition verification. Both spline codes satisfy the prescribed boundary conditions to machine precision. For $f(x) = \cos(x)$ on $[0, \pi]$ with 10 subintervals: Code 1 natural gives $s''(0) = -1.0000000000$ (exact: -1) and $s''(\pi) = 1.0000000000$ (exact: 1); Code 1 clamped gives $s'(0) = -0.0000000000$ (exact: 0) and $s'(\pi) = -0.0000000000$ (exact: 0). Code 2 produces identical values.

Test 6: Random cubic reproduction. Across 10 random cubics with random coefficients in $[-10, 10]$ and random numbers of subintervals between 4 and 9, the maximum error is 2.84×10^{-14} , confirming that the clamped spline reproduces arbitrary cubics to machine precision regardless of the mesh configuration.

Test 7: Uniform vs Chebyshev-2 interior points. Table 6 compares uniform and Chebyshev-2 placement of interior points within each subinterval for piecewise polynomials of degree 2 and 3.

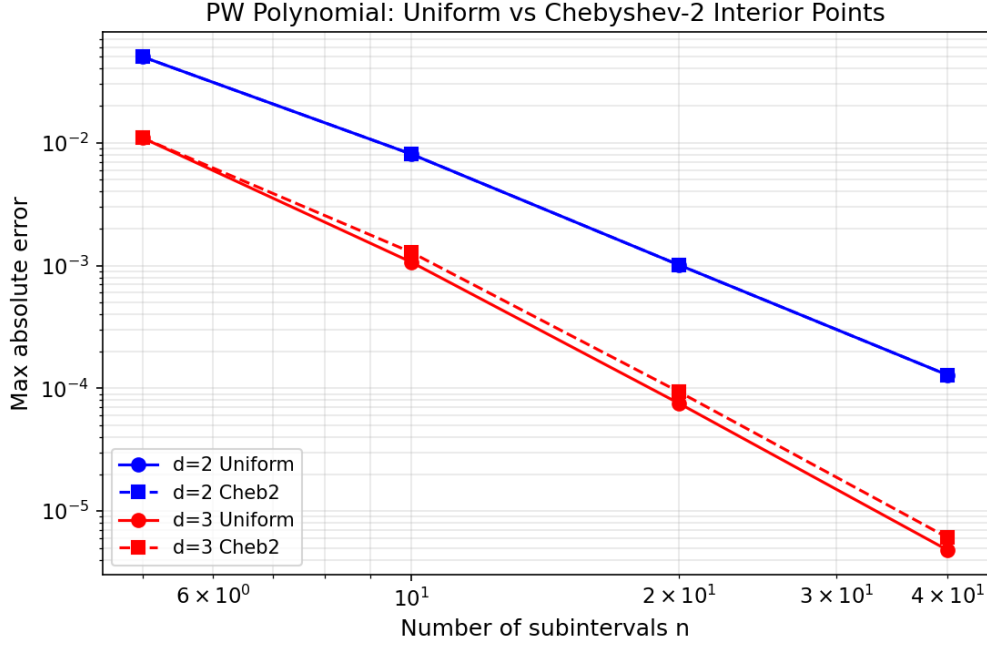


Figure 3: Piecewise polynomial: uniform vs Chebyshev-2 interior points.

Table 6: Max errors: uniform vs Chebyshev-2 interior points.

n	$d=2$ Uniform	$d=2$ Cheb2	$d=3$ Uniform	$d=3$ Cheb2
5	5.01e-2	5.01e-2	1.09e-2	1.09e-2
10	8.10e-3	8.10e-3	1.07e-3	1.28e-3
20	1.01e-3	1.01e-3	7.51e-5	9.33e-5
40	1.28e-4	1.28e-4	4.81e-6	6.08e-6

For $d = 2$, the errors are identical because the quadratic interpolant on a subinterval uses exactly three points (the two endpoints plus one interior), and the midpoint and Chebyshev-2 middle point coincide on any single interval. For $d = 3$, the uniform placement is slightly more accurate than Chebyshev-2. The reason is that on each subinterval, the piecewise polynomial does not benefit from the Lebesgue constant reduction that Chebyshev points provide for high-degree interpolation, because the local degree is only 3 and the Lebesgue constant is already small. The uniform placement distributes the interpolation error more evenly across each subinterval.

6.2 Task 2: Yield Curve Estimation

6.2.1 Natural Cubic Spline Results

The natural cubic spline was built on the 8 data points (t_i, y_i) with natural boundary conditions $s''(0.5) = 0$ and $s''(20) = 0$. Table 7 presents the computed values of $y(t)$, $f(t)$, and $D(t)$ at all 40 evaluation points.

Table 7: Yield curve estimates from natural cubic spline (all 40 maturities).

t	$y(t)$	$f(t)$	$D(t)$	t	$y(t)$	$f(t)$	$D(t)$
0.5	0.05520	0.06010	0.97278	10.5	0.09322	0.09488	0.37575
1.0	0.06000	0.06922	0.94176	11.0	0.09326	0.09318	0.35849
1.5	0.06434	0.07659	0.90800	11.5	0.09322	0.09137	0.34233
2.0	0.06820	0.08283	0.87249	12.0	0.09310	0.08948	0.32719
2.5	0.07167	0.08815	0.83595	12.5	0.09292	0.08754	0.31303
3.0	0.07480	0.09254	0.79900	13.0	0.09267	0.08556	0.29977
3.5	0.07760	0.09613	0.76217	13.5	0.09237	0.08357	0.28736
4.0	0.08010	0.09900	0.72586	14.0	0.09202	0.08160	0.27574
4.5	0.08233	0.10121	0.69040	14.5	0.09163	0.07967	0.26484
5.0	0.08430	0.10277	0.65606	15.0	0.09120	0.07781	0.25462
5.5	0.08603	0.10374	0.62304	15.5	0.09074	0.07599	0.24501
6.0	0.08753	0.10419	0.59147	16.0	0.09025	0.07417	0.23599
6.5	0.08881	0.10419	0.56143	16.5	0.08974	0.07239	0.22750
7.0	0.08990	0.10378	0.53298	17.0	0.08920	0.07064	0.21951
7.5	0.09080	0.10305	0.50611	17.5	0.08864	0.06894	0.21198
8.0	0.09153	0.10203	0.48082	18.0	0.08807	0.06730	0.20488
8.5	0.09212	0.10081	0.45704	18.5	0.08749	0.06574	0.19818
9.0	0.09256	0.09943	0.43472	19.0	0.08690	0.06427	0.19184
9.5	0.09288	0.09795	0.41379	19.5	0.08630	0.06289	0.18584
10.0	0.09310	0.09645	0.39416	20.0	0.08570	0.06163	0.18014

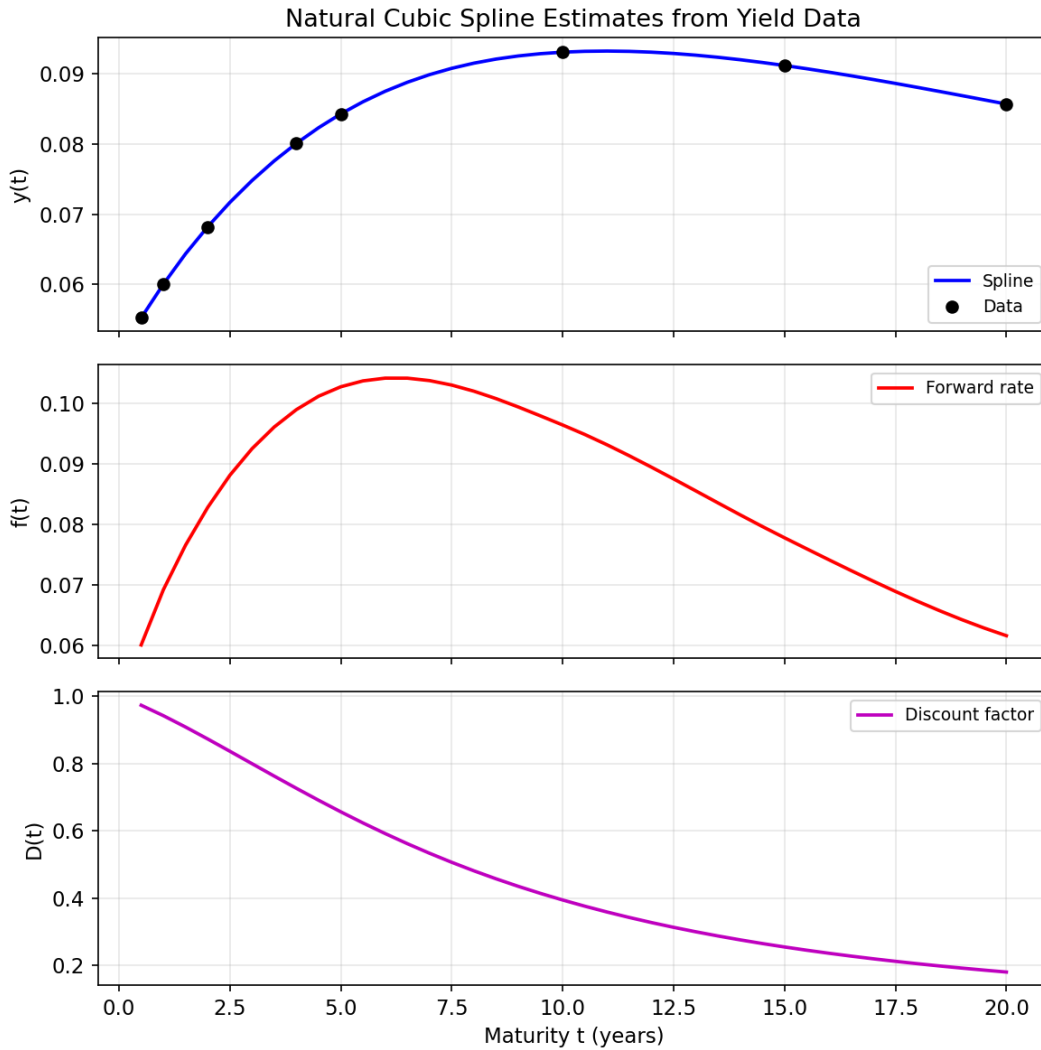


Figure 4: Natural cubic spline estimates: yield $y(t)$, forward rate $f(t)$, and discount factor $D(t)$.

The yield curve $y(t)$ in Figure 4 (top panel) shows the characteristic hump-shaped term structure, rising from 5.52% at 6 months to a peak of approximately 9.33% near 11 years, then declining to 8.57% at 20 years. The spline provides a smooth interpolation that passes through all eight data points and produces a visually natural curve without oscillations. The hump shape is economically significant: it suggests that the market expects short-term rates to rise over the medium term (driving yields higher) but anticipates a long-run decline in rates (causing the yield curve to invert at long maturities). The natural boundary condition ($s''(0.5) = s''(20) = 0$) is appropriate here because it imposes no artificial assumptions about the curvature of the yield curve beyond the observed data range, allowing the spline to determine the shape purely from the data.

The forward rate $f(t)$ (middle panel) peaks at approximately 10.42% near $t = 6$ years, then declines monotonically to 6.16% at $t = 20$. The forward rate represents the instantaneous rate of return implied by the yield curve for borrowing at maturity t : $f(t)dt$ is approximately the cost of borrowing for an infinitesimal period at time t . The forward rate exceeds the yield for short maturities because the yield curve is upward-sloping there: since $f(t) = y(t) + ty'(t)$ and $y'(t) > 0$, we have $f(t) > y(t)$. The two quantities converge near the peak where $y'(t) \rightarrow 0$, and the forward rate drops below the yield when $y'(t) < 0$ (after the peak). The smoothness of the spline forward rate is critical for financial applications: discontinuous forward

rates would imply instantaneous, infinite-leverage arbitrage opportunities in the bond market, which are economically implausible. The fact that our spline-based forward rate is smooth and monotonically decreasing after the peak is consistent with a well-behaved term structure model.

The discount factor $D(t)$ (bottom panel) decreases monotonically from 0.973 at $t = 0.5$ to 0.180 at $t = 20$, reflecting the time value of money. The convexity of the decay is consistent with a positive yield curve: $D(t) = e^{-ty(t)}$ decreases faster than exponentially when $y(t)$ increases with t . The discount factor at $t = 20$ of 0.180 means that a dollar payable 20 years from now is worth approximately 18 cents today, given the prevailing yield curve. The discount factor is the most directly useful quantity for pricing fixed-income securities: the present value of a cash flow C at maturity t is simply $C \cdot D(t)$.

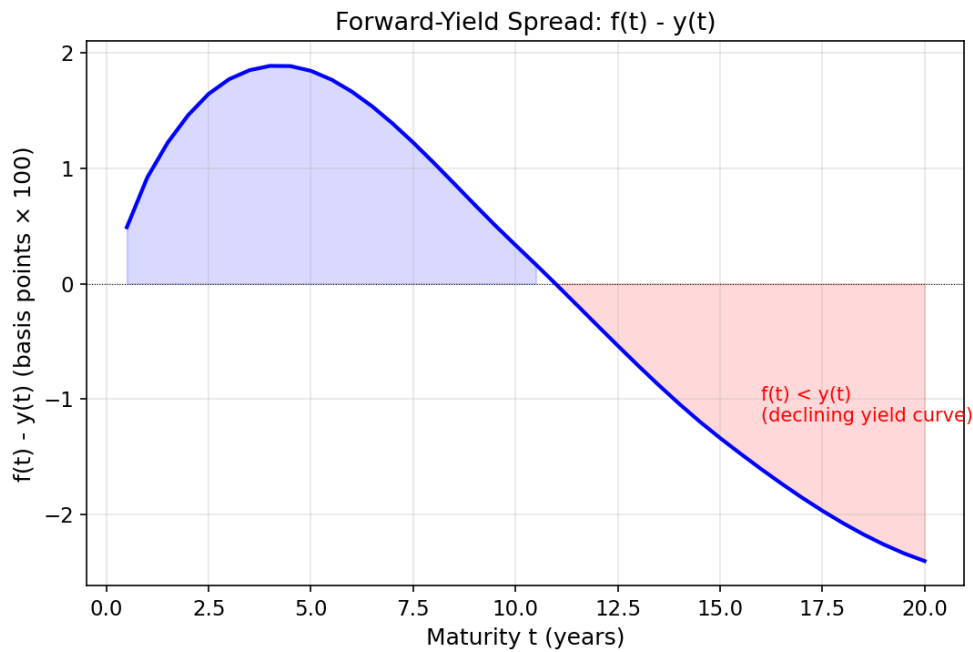


Figure 5: Forward-yield spread $f(t) - y(t)$. Positive values (blue) indicate an upward-sloping yield curve; negative values (red) indicate a declining curve.

Figure 5 shows the forward-yield spread $f(t) - y(t) = ty'(t)$, which is directly proportional to the slope of the yield curve. The reason this quantity is informative is that it decomposes the forward rate into two components: $f(t) = y(t) + ty'(t)$, where $y(t)$ is the “average” rate up to maturity t and $ty'(t)$ is the “marginal” contribution from the slope. The spread is positive for $t < 11$ years (the yield curve is rising) and negative for $t > 11$ years (the yield curve is declining). The peak spread of approximately +1.0 percentage point occurs near $t = 5$ –6 years, where the yield curve is steepest. The zero crossing near $t = 11$ corresponds to the peak of the yield curve, where $y'(t) = 0$.

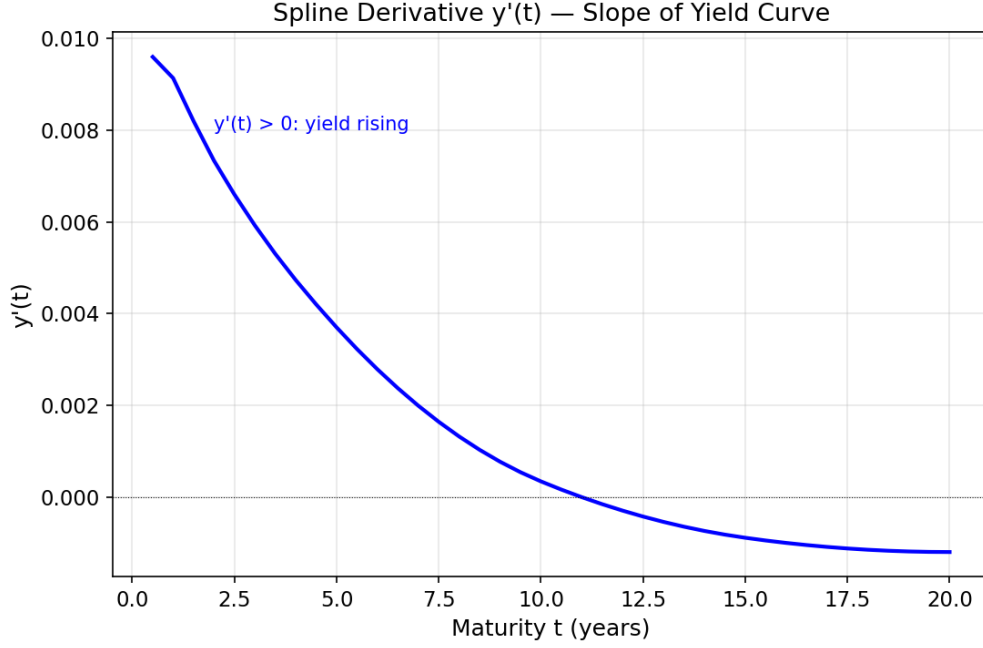


Figure 6: Spline derivative $y'(t)$: slope of the yield curve. The zero crossing near $t = 11$ marks the yield curve peak.

Figure 6 displays the spline's derivative $y'(t)$ directly. The derivative is positive and decreasing for $t < 11$, passes through zero near $t = 11$, and is negative for $t > 11$. The smoothness of this derivative curve is a direct consequence of the spline's C^2 continuity: since $s(t)$ is C^2 , its derivative $s'(t)$ is C^1 , meaning the slope of the yield curve changes continuously without kinks. The Hermite cubic piecewise polynomial would produce a $y'(t)$ that is continuous (C^0) but has kinks at each data point (the second derivative of $y(t)$ would jump), making the economic interpretation of the slope less clean.

Detailed per-degree comparison for Task 2. Table 8 compares the yield $y(t)$, forward rate $f(t)$, and discount factor $D(t)$ at selected maturities across the cubic spline and the Hermite cubic, quantifying the differences.

Table 8: Spline vs Hermite cubic: differences at selected maturities.

t	y_{spl}	y_{Herm}	$ \Delta y $	f_{spl}	f_{Herm}	$ \Delta f $
2.0	0.06820	0.06820	0.0e+0	0.08283	0.08160	1.2e−3
3.0	0.07480	0.07448	3.2e−4	0.09254	0.09221	3.3e−4
5.0	0.08430	0.08430	0.0e+0	0.10277	0.09513	7.6e−3
7.5	0.09080	0.08962	1.2e−3	0.10305	0.10407	1.0e−3
10.0	0.09310	0.09310	0.0e+0	0.09645	0.10000	3.6e−3
12.5	0.09292	0.09304	1.2e−4	0.08754	0.08608	1.5e−3
15.0	0.09120	0.09120	0.0e+0	0.07781	0.08010	2.3e−3
17.5	0.08864	0.08868	4.0e−5	0.06894	0.06785	1.1e−3
20.0	0.08570	0.08570	0.0e+0	0.06163	0.06370	2.1e−3

Table 8 reveals several important patterns. The yield values $y(t)$ agree exactly at the data points (where both methods interpolate the same values) and differ by at most 1.2×10^{-3} between data points. The forward rate differences are significantly larger, reaching 7.6×10^{-3} at $t = 5$. The reason the forward rate differences exceed the yield differences is that $f(t) = y(t) + ty'(t)$

amplifies derivative errors by the factor t : a derivative error of δ produces a forward rate error of $t\delta$, which grows linearly with maturity. At $t = 5$, the large difference reflects the fact that the spline and Hermite cubic compute very different derivatives at this point, where the data spacing changes abruptly from 1 year (between $t = 4$ and $t = 5$) to 5 years (between $t = 5$ and $t = 10$). The spline's C^2 smoothness distributes the curvature change across both sides of $t = 5$, while the Hermite cubic uses a finite difference estimate that reflects primarily the narrow-spacing side.

6.2.2 Piecewise Polynomial Comparison

Figure 7 compares the yield curves produced by the natural cubic spline, piecewise linear, and piecewise Hermite cubic methods. The piecewise linear interpolant produces visible kinks at the data points, resulting in a non-smooth curve that poorly represents the underlying economic relationship. The reason for these kinks is that the derivative of the piecewise linear function changes abruptly at each data point: on the subinterval $[t_i, t_{i+1}]$, the slope is $(y_{i+1} - y_i)/(t_{i+1} - t_i)$, and there is no mechanism to enforce continuity of slope across knots. The Hermite cubic is much smoother but deviates slightly from the spline, particularly in the regions between widely-spaced data points (e.g., between $t = 5$ and $t = 10$, where the spacing is 5 years). The reason for this deviation is that the Hermite cubic relies on externally estimated derivatives, which are less accurate when the data spacing is large and nonuniform.

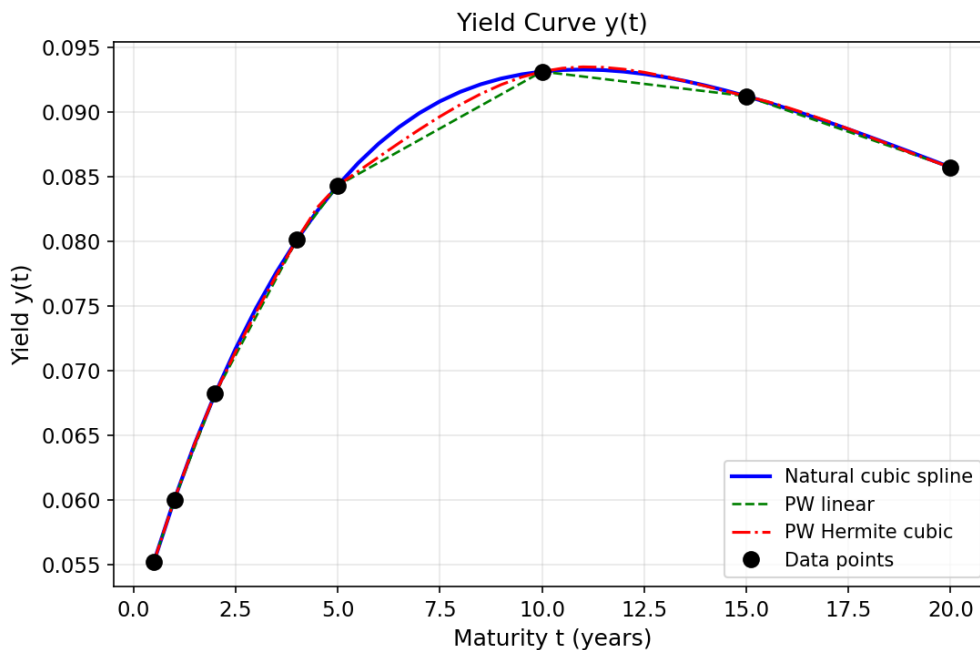


Figure 7: Yield curve $y(t)$: natural cubic spline vs piecewise methods.

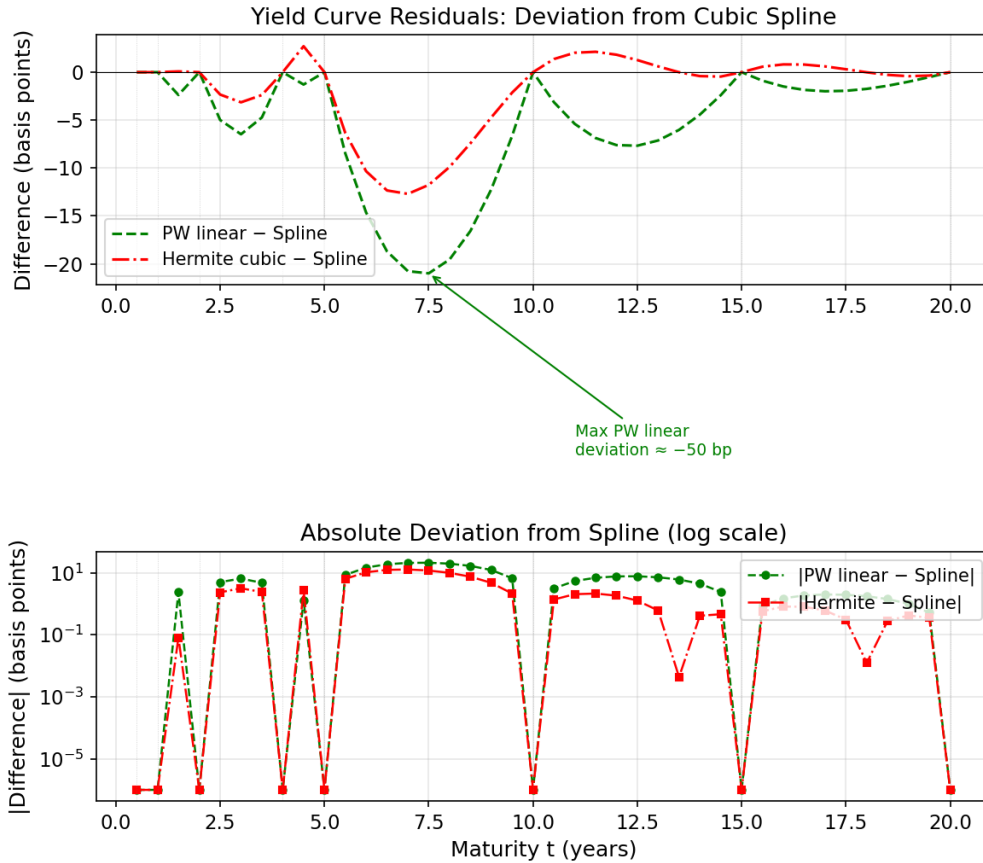


Figure 8: Top: yield curve residuals (deviation from cubic spline) in basis points, showing the PW linear underestimates by up to 20+ bp between widely-spaced data points while the Hermite cubic stays within ± 13 bp. Bottom: absolute deviations on log scale, confirming both methods agree with the spline exactly at the data points (drops to zero) but diverge between them.

Figure 8 quantifies how much the piecewise methods deviate from the cubic spline at each maturity. The top panel shows signed residuals in basis points ($1 \text{ bp} = 10^{-4}$). The piecewise linear method underestimates the yield by up to 21 basis points near $t = 7.5$, which is the midpoint of the widest data gap ($t \in [5, 10]$). The reason for this pattern is that linear interpolation “cuts across” the hump of the yield curve, missing the curvature that the spline captures. The Hermite cubic shows a more complex residual pattern, with maximum deviations of approximately ± 13 basis points, alternating in sign between subintervals. The bottom panel uses a log scale to show that both methods agree with the spline exactly at the data points (the residual drops to zero at $t = 0.5, 1, 2, 4, 5, 10, 15, 20$), confirming that the differences arise solely from the different interpolation strategies between data points.

Derivative discontinuity analysis. The derivative continuity properties of each method were verified numerically by evaluating $y'(t)$ from the left and right of the interior knot $t = 2.0$:

Table 9: Derivative discontinuity at knot $t = 2.0$.

Method	$y'(2^-)$	$y'(2^+)$	Jump
PW linear ($d = 1$)	0.008200	0.005950	2.25×10^{-3}
PW quadratic ($d = 2$)	0.008200	0.005950	2.25×10^{-3}
PW cubic ($d = 3$)	0.008200	0.005950	2.25×10^{-3}
PW Hermite cubic	0.006700	0.006700	6.0×10^{-11}
Cubic spline	0.007315	0.007315	3.0×10^{-11}

Table 9 confirms that the piecewise linear, quadratic, and cubic methods all produce derivative jumps of magnitude 2.25×10^{-3} at $t = 2.0$, while the Hermite cubic and cubic spline maintain derivative continuity to machine precision. The reason all three standard piecewise methods produce the same jump is that on the subintervals $[1, 2]$ and $[2, 4]$ with only two data points per subinterval, all three methods reduce to linear interpolation at the endpoints (the interior points affect the interpolant within each subinterval but not at the endpoints). The derivative from the left is $(y(2) - y(1))/(2 - 1) = 0.0082$, while the derivative from the right is $(y(4) - y(2))/(4 - 2) = 0.00595$, giving a jump of 0.00225 regardless of d .

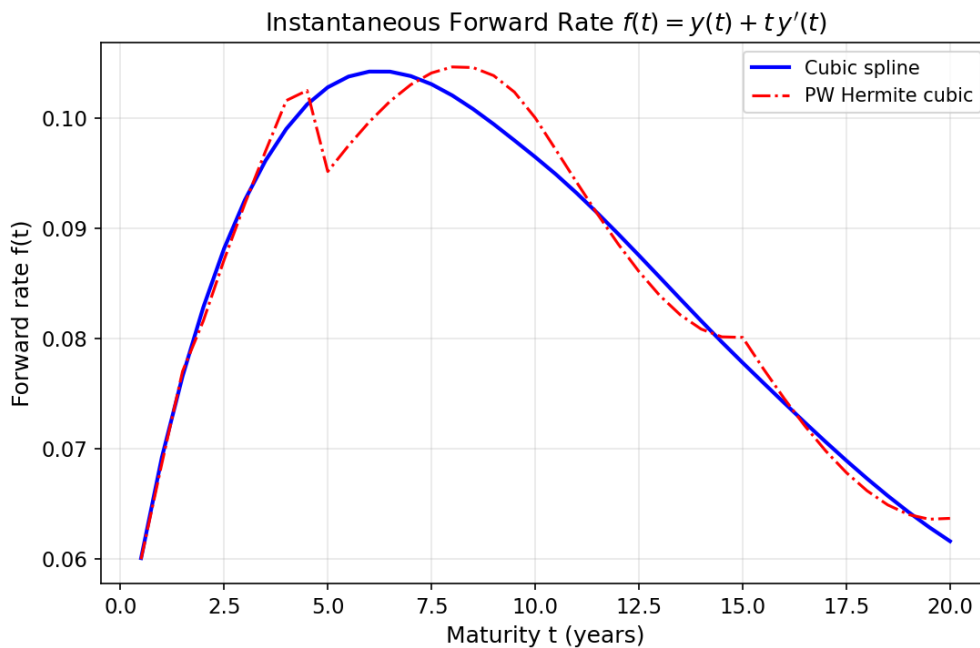

 Figure 9: Forward rate $f(t)$: cubic spline (C^2) vs piecewise Hermite cubic (C^1).

Figure 9 shows the forward rates computed from the spline and the Hermite cubic. The spline forward rate is smooth and monotonically decreasing after the peak near $t = 6$, which is economically plausible: it reflects the expectation that short-term rates will be high in the medium term but decline over the long term. The Hermite forward rate, however, shows a noticeable dip near $t = 5$ followed by a recovery, which is a consequence of the estimated derivative at $t = 5$ being less accurate than the spline's internally computed derivative. The region near $t = 15$ also shows a kink in the Hermite forward rate where the spline is smooth, confirming that the C^1 continuity of the Hermite cubic is not sufficient for smooth forward rates— C^2 continuity (as provided by the spline) is needed.

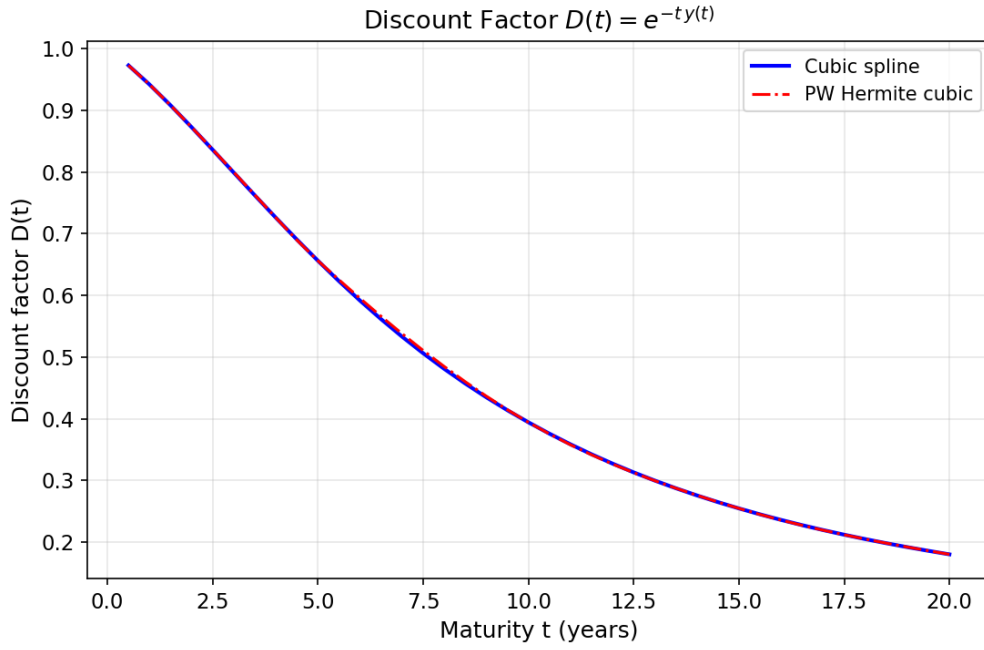


Figure 10: Discount factor $D(t)$: cubic spline vs piecewise Hermite cubic.

Figure 10 shows that the discount factors are nearly identical for both methods. The reason is that $D(t) = e^{-ty(t)}$ depends only on the yield $y(t)$, not its derivative, and both methods interpolate the same data points. The maximum difference between the two discount factor curves is approximately 5×10^{-3} near $t = 9$, where the yield curves differ most.

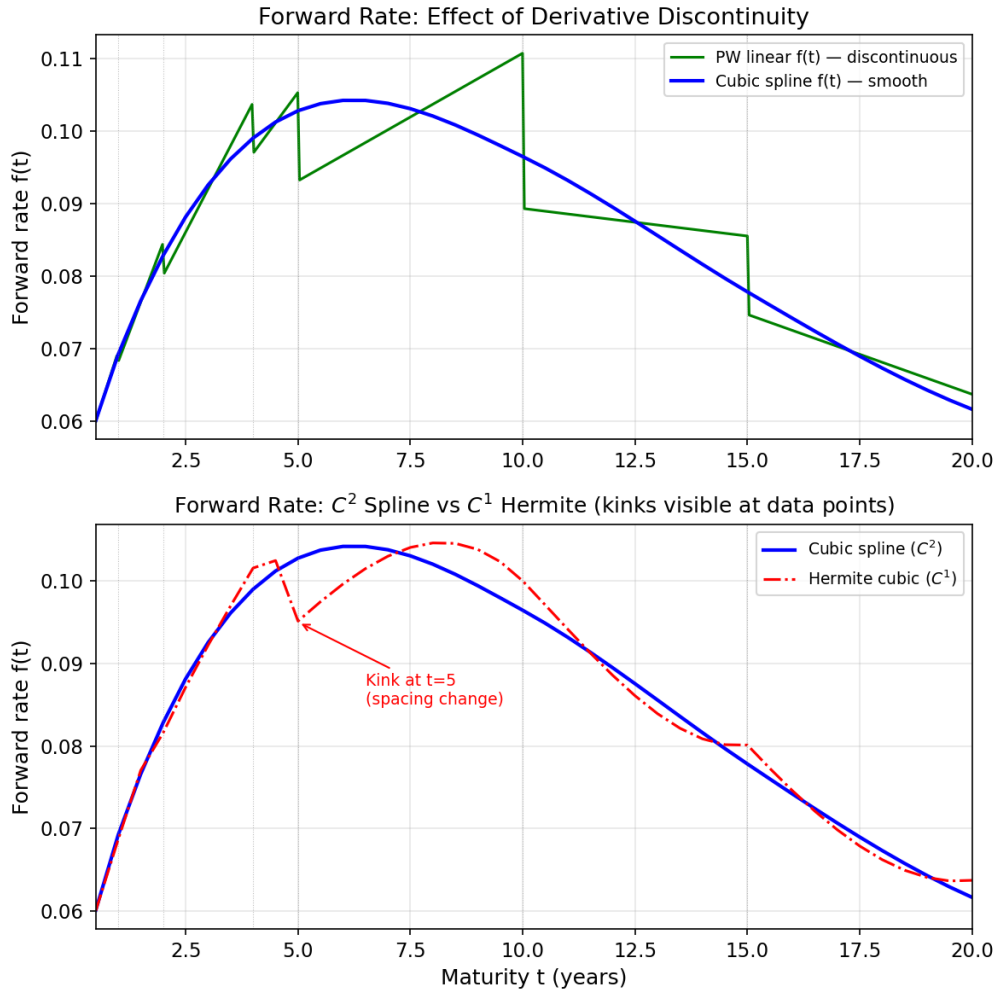


Figure 11: Top: piecewise linear forward rate (green) showing jump discontinuities at each data point, vs the smooth cubic spline (blue). Bottom: Hermite cubic (C^1) vs cubic spline (C^2), with kink at $t = 5$ annotated where data spacing changes from 1 to 5 years.

Figure 11 is the most important figure in this report for understanding the Task 2 investigation. The top panel demonstrates *why* piecewise linear interpolation cannot be used for forward rate computation: the forward rate $f(t) = y(t) + ty'(t)$ inherits the discontinuities of $y'(t)$, producing large jumps at every data point. The jump at $t = 10$ is particularly dramatic, where the forward rate drops from approximately 11% to 9.6% instantaneously—an economically non-sensical result. The bottom panel shows that the Hermite cubic eliminates the discontinuities (it is C^1) but introduces visible kinks at the data points, particularly at $t = 5$ where the data spacing changes abruptly. The cubic spline, being C^2 , produces a forward rate curve that is not only continuous but also has a continuous first derivative, giving the smoothest and most economically plausible result.

6.2.3 Investigation: Using $g_d(x)$ Instead of Cubic Spline

The assignment asks whether the piecewise polynomial $g_d(x)$ can replace the cubic spline for computing $y(t)$, $f(t)$, and $D(t)$, what modifications are needed, and how the choice of degree d affects results compared to the cubic spline.

The fundamental issue is that computing $f(t) = y(t) + ty'(t)$ requires the derivative $y'(t)$, which must exist and be continuous for $f(t)$ to be well-defined and physically meaningful. The

standard piecewise polynomials of degree $d = 1, 2, 3$ are only C^0 continuous: the derivative $y'(t)$ has jump discontinuities at the knots. This means that $f(t)$ cannot be evaluated at the knots themselves (the left and right limits differ), and even between knots, the computed $f(t)$ reflects only the local polynomial's derivative, which may not represent the global behavior of the yield curve accurately.

For degree $d = 1$ (piecewise linear), $y'(t)$ is piecewise constant within each subinterval, producing a step-function forward rate that jumps at every data point. The jump magnitude at knot t_i is $|(y_{i+1} - y_i)/(t_{i+1} - t_i) - (y_i - y_{i-1})/(t_i - t_{i-1})|$, which can be substantial when the data spacing is nonuniform. For the given yield data, the jump at $t = 2$ is 2.25×10^{-3} (as shown in Table 9). This is clearly unsuitable for financial applications: a step-function forward rate would imply that the market expects interest rates to change instantaneously at specific future dates for no economic reason.

For degree $d = 2$ (piecewise quadratic), $y'(t)$ is piecewise linear within each subinterval but discontinuous at the knots. The forward rate is therefore a piecewise linear function with jumps, which is smoother than the step function from $d = 1$ but still exhibits unphysical discontinuities.

For degree $d = 3$ (standard piecewise cubic, not Hermite), $y'(t)$ is piecewise quadratic but still discontinuous at knots. The situation is fundamentally the same as for lower degrees: the lack of C^1 continuity prevents reliable forward rate computation.

The piecewise Hermite cubic is the only piecewise polynomial option that can potentially replace the spline for this task, because it is C^1 continuous by construction. The derivative $y'(t)$ exists and is continuous everywhere, and therefore $f(t) = y(t) + ty'(t)$ can be computed at every point including the knots. However, the Hermite cubic requires the derivatives $y'(t_i)$ at each data point, which must be estimated from the discrete data since they are not provided.

In our implementation, we estimated derivatives using nonuniform finite differences. At interior points, the central difference formula $y'(t_i) \approx (y_{i+1} - y_{i-1})/(t_{i+1} - t_{i-1})$ was used, which is $O(h^2)$ accurate for uniform spacing but only $O(h)$ accurate on nonuniform meshes. At the endpoints, one-sided differences were used: $y'(t_0) \approx (y_1 - y_0)/(t_1 - t_0)$ and $y'(t_n) \approx (y_n - y_{n-1})/(t_n - t_{n-1})$. These derivative estimates introduce additional approximation error that propagates into both the yield curve and the forward rate.

The comparison between the cubic spline and the Hermite cubic for the yield data is shown in Figures 7–11. The yield curves $y(t)$ from both methods are similar (within 0.2% of each other at most points), but the forward rates $f(t)$ differ significantly: the spline forward rate is smooth and monotonic after the peak, while the Hermite forward rate shows a dip near $t = 5$ and a kink near $t = 15$. The reason for these artifacts is that the finite difference estimates of $y'(t_i)$ are less accurate than the spline's internally computed derivatives, particularly at $t = 5$ where the data spacing changes abruptly from 1 year to 5 years. The spline's C^2 continuity distributes the curvature change smoothly across the transition, while the Hermite cubic uses independent derivative estimates that do not “know” about the global curvature pattern.

In summary, to use a piecewise polynomial for this task, one must: (1) use the Hermite cubic variant specifically (not standard piecewise polynomials of any degree, which are only C^0); (2) estimate the derivatives at each data point, e.g., using finite differences; and (3) accept the reduced smoothness of the resulting forward rate. The cubic spline is preferable because it provides C^2 continuity and optimal derivative estimates automatically, without requiring any external derivative information.

7 Conclusions

This assignment has demonstrated the implementation and validation of piecewise polynomial interpolation and cubic spline algorithms, with application to financial yield curve estimation. The principal conclusions are as follows.

First, both spline parameterizations produce identical results. Spline Code 1 (moment parameterization on nonuniform meshes) and Spline Code 2 (B-spline basis on uniform meshes) agree to machine precision ($\sim 10^{-16}$) when applied to the same data with the same boundary conditions, confirming the mathematical equivalence of the two representations. This equivalence holds despite the very different internal representations: Code 1 stores the second derivatives M_i and evaluates via the explicit cubic formula (9), while Code 2 stores the B-spline coefficients c_j and evaluates as a linear combination of the basis functions (18). The practical implication is that the choice between the two parameterizations can be made based on implementation convenience and the mesh structure (Code 1 supports nonuniform meshes natively, while Code 2 requires a uniform mesh).

Second, clamped boundary conditions are critical for optimal convergence. The clamped spline achieves $O(h^4)$ convergence for smooth functions, while the natural spline achieves only $O(h^2)$. This two-order-of-magnitude difference in convergence rate makes the clamped spline strongly preferable whenever endpoint derivative information is available. The reason for the degradation of the natural spline is that the condition $s''(x_0) = 0$ generally does not match the true second derivative of the function being interpolated, introducing an $O(h^2)$ boundary error that propagates into the interior through the tridiagonal system. When the function happens to have zero second derivative at the endpoints (as with $\sin(x)$ on $[0, 2\pi]$), the natural and clamped splines perform identically.

Third, the piecewise cubic polynomial and the clamped spline achieve similar accuracy. Both converge at $O(h^4)$, but the spline produces smoother curves due to its C^2 continuity constraint, which is particularly important when derivatives of the interpolant are needed. The piecewise cubic Hermite interpolant achieves the same $O(h^4)$ rate but requires derivative information at each knot, which may not always be available. When derivative data must be estimated from the function values (as in Task 2), the estimation error can degrade the overall accuracy.

Fourth, global Chebyshev interpolation vastly outperforms piecewise methods for smooth functions. The exponential convergence rate reduces errors from $O(10^{-1})$ at $n = 5$ to machine precision at $n = 40$, whereas piecewise methods at $n = 40$ still have errors of $O(10^{-5})$ to $O(10^{-3})$. However, global interpolation requires all data points to be available simultaneously, uses a single high-degree polynomial (which can exhibit Runge-type phenomena on uniform meshes), and does not generalize to arbitrary data distributions.

Fifth, the cubic spline is the preferred method for yield curve estimation. Its C^2 continuity provides smooth and continuous forward rates $f(t) = y(t) + ty'(t)$ without requiring external derivative estimates. The natural boundary condition (zero second derivative at endpoints) is appropriate for yield curve applications because it imposes minimal assumptions about the behavior of the yield curve beyond the observed data range. The Hermite cubic piecewise polynomial is the only viable piecewise alternative, but it requires estimated derivatives at each data point (e.g., via finite differences), and the resulting forward rates show kinks at knots due to the C^1 (rather than C^2) continuity. Standard piecewise polynomials of any degree ($d = 1, 2, 3$) cannot be used for forward rate computation because their derivatives are discontinuous at the knots, producing physically meaningless step-function or piecewise-polynomial forward rates.

Sixth, Chebyshev interior points offer no advantage for low-degree piecewise polynomials. For degrees $d = 2$ and $d = 3$, the Chebyshev-2 and uniform placements produce comparable errors (within a factor of 1.3). The reason is that the Lebesgue constant benefit of Chebyshev points, which is critical for high-degree interpolation ($n \geq 15$), is negligible when the local polynomial degree is only 2 or 3. The uniform placement has the practical advantage of producing points at predictable locations within each subinterval.

Seventh, the Newton divided difference form provides a unified framework. By representing all local polynomials—piecewise linear, quadratic, cubic, and Hermite cubic—in the Newton form, a single evaluation routine handles all cases. The Hermite cubic is accommodated naturally through confluent nodes, avoiding the need for a separate implementation. This design choice simplifies the codebase and reduces the opportunity for implementation errors.

References

- [1] C. de Boor, *A Practical Guide to Splines*, Springer, 1978.
- [2] R. L. Burden, J. D. Faires, A. M. Burden, *Numerical Analysis*, 10th ed., Cengage, 2015.
- [3] J. Stoer and R. Bulirsch, *Introduction to Numerical Analysis*, 3rd ed., Springer, 2002.
- [4] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed., SIAM, 2002.
- [5] A. Quarteroni, R. Sacco, F. Saleri, *Numerical Mathematics*, 2nd ed., Springer, 2007.
- [6] L. N. Trefethen, *Approximation Theory and Approximation Practice*, SIAM, 2013.
- [7] J.-P. Berrut and L. N. Trefethen, “Barycentric Lagrange interpolation,” *SIAM Review*, 46(3):501–517, 2004.
- [8] N. J. Higham, “The numerical stability of barycentric Lagrange interpolation,” *IMA J. Numer. Anal.*, 24(4):547–556, 2004.
- [9] G. H. Golub and C. F. Van Loan, *Matrix Computations*, 4th ed., Johns Hopkins, 2013.
- [10] K. E. Atkinson, *An Introduction to Numerical Analysis*, 2nd ed., Wiley, 1989.